UTS Deep Learning (Kelas B) — Prediksi Trajektori Bola Basket

Nama: KHAERUL HADISWARA
NIM: 25917025
Mata Kuliah: Deep Learning
Prodi/Kelas: Magister Informatika / B
Dosen: Ir. Chandra Kusuma Dewa, S.Kom., M.Cs., Ph.D.
Tanggal Ujian: Jumat, 5 Juni 2026
Sifat: Take Home (Deadline: 12 Juni 2026)

Deskripsi UTS

Membangun model Deep Learning yang mampu memprediksi trajektori posisi koordinat X dan Y bola basket di masa depan berdasarkan pola pergerakan beberapa frame sebelumnya.

Daftar Isi

No	Topik	Bobot
1	Ekstraksi & YOLO Tracking	15%
2	Data Engineering - CSV Exporter	10%
3	Data Engineering - Train/Test Split Kronologis	10%
4	Data Engineering - Sliding Window	15%
5	Arsitektur Model (Simple RNN, LSTM, GRU)	20%
6	Evaluasi Proses Training	10%
7	Pengujian & Komparasi Trajektori	20%

⚙️ Setup & Import Library

In [1]:
```
# ═══════════════════════════════════════════════════════════════════
# SETUP AWAL: Import library dan konfigurasi global
# ═══════════════════════════════════════════════════════════════════
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings
import time

warnings.filterwarnings('ignore')
matplotlib.rcParams['figure.dpi'] = 120
matplotlib.rcParams['figure.figsize'] = (12, 6)
plt.style.use('seaborn-v0_8-whitegrid')

# Buat folder output
for folder in ['output', 'figures', 'models']:
    os.makedirs(folder, exist_ok=True)

print("=" * 60)
print("ENVIRONMENT CHECK")
print("=" * 60)
```

```
print(f"  OpenCV     : {cv2.__version__}")
print(f"  NumPy      : {np.__version__}")
print(f"  Pandas     : {pd.__version__}")
print(f"  Matplotlib : {matplotlib.__version__}")

# Cek YOLO
try:
    from ultralytics import YOLO
    import ultralytics
    print(f"  Ultralytics: {ultralytics.__version__}")
except ImportError:
    print("  Ultralytics: BELUM TERINSTALL! Jalankan: pip install ultralytics")

# Cek TensorFlow/Keras
import tensorflow as tf
print(f"  TensorFlow : {tf.__version__}")
print(f"  GPU        : {tf.config.list_physical_devices('GPU') or 'CPU Only'}")

print("\n✅ Semua library siap digunakan.")
```

```
============================================================
ENVIRONMENT CHECK
============================================================
  OpenCV     : 4.13.0
  NumPy      : 2.2.6
  Pandas     : 2.3.3
  Matplotlib : 3.10.7
  Ultralytics: 8.4.60
  TensorFlow : 2.20.0
  GPU        : CPU Only

✅ Semua library siap digunakan.
```

📼 Persiapan Video — Pemotongan 3 Menit

Sesuai instruksi soal, video asli dipotong menjadi **3 menit** dengan mengambil **bagian tengah** video di mana objek bola basket terlihat jelas dan aktif.

- **Video asli:** `DEEP LEARNING VIDEO.mp4`
- **Durasi potong:** 3 menit (180 detik = 5.400 frame)
- **Posisi potong:** Menit 3:30 s.d. 6:30 (tengah video)

In [6]:
```
import cv2
import os

VIDEO_INPUT = "DEEP LEARNING VIDEO.mp4"

cap = cv2.VideoCapture(VIDEO_INPUT)
print("isOpened:", cap.isOpened())
print("fps:", cap.get(cv2.CAP_PROP_FPS))
print("frame_count:", cap.get(cv2.CAP_PROP_FRAME_COUNT))
print("width:", cap.get(cv2.CAP_PROP_FRAME_WIDTH))
print("height:", cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
cap.release()

print("file_exists:", os.path.exists(VIDEO_INPUT))
print("file_size:", os.path.getsize(VIDEO_INPUT))
```

```
isOpened: True
fps: 30.0
frame_count: 36003.0
width: 1280.0
height: 720.0
file_exists: True
file_size: 138698087
```

In [7]:
```
# PEMOTONGAN VIDEO: Ambil 3 menit bagian tengah

import cv2
import os
```

```
VIDEO_INPUT = "DEEP LEARNING VIDEO.mp4"
VIDEO_OUTPUT = "output/video_3menit.mp4"

cap = cv2.VideoCapture(VIDEO_INPUT)
if not cap.isOpened():
    raise ValueError("Video input gagal dibuka")

fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("fps:", fps)
print("total_frames:", total_frames)
print("size:", width, height)

start_sec = 3 * 60 + 30
duration_sec = 180
start_frame = int(start_sec * fps)
end_frame = start_frame + int(duration_sec * fps)

print("start_frame:", start_frame)
print("end_frame:", end_frame)

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter(VIDEO_OUTPUT, fourcc, fps, (width, height))

if not writer.isOpened():
    raise ValueError("VideoWriter gagal dibuka")

frame_idx = 0
saved = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_idx += 1

    if frame_idx < start_frame:
        continue
    if frame_idx >= end_frame:
        break

    if frame is None or frame.size == 0:
        print("frame kosong pada:", frame_idx)
        continue

    if frame.shape[1] != width or frame.shape[0] != height:
        frame = cv2.resize(frame, (width, height))

    writer.write(frame)
    saved += 1

cap.release()
writer.release()

print("saved_frames:", saved)
print("output_exists:", os.path.exists(VIDEO_OUTPUT))
print("output_size:", os.path.getsize(VIDEO_OUTPUT) if os.path.exists(VIDEO_OUTPUT)
```

```
fps: 30.0
total_frames: 36003
size: 1280 720
start_frame: 6300
end_frame: 11700
saved_frames: 5400
output_exists: True
output_size: 42654114
```

In [8]:

```
from ultralytics import YOLO
import cv2
```

```python
import os

VIDEO_PATH = "output/video_3menit.mp4"
model = YOLO("yolov8x.pt")  # untuk debug, lebih kuat dari yolov8s

cap = cv2.VideoCapture(VIDEO_PATH)
sample_frames = [300, 800, 1200, 1800, 2500, 3200, 4000, 5000]

os.makedirs("debug_frames", exist_ok=True)

for fno in sample_frames:
    cap.set(cv2.CAP_PROP_POS_FRAMES, fno)
    ret, frame = cap.read()
    if not ret:
        print(f"Frame {fno}: gagal dibaca")
        continue

    results = model.predict(
        source=frame,
        imgsz=1280,
        conf=0.05,
        verbose=False
    )

    r = results[0]
    print(f"\nFrame {fno}")
    if r.boxes is None or len(r.boxes) == 0:
        print("  Tidak ada deteksi")
    else:
        for i in range(len(r.boxes)):
            cls_id = int(r.boxes.cls[i].item())
            cls_name = model.names[cls_id]
            conf = float(r.boxes.conf[i].item())
            print(f"  {cls_name} | conf={conf:.3f}")

    annotated = r.plot()
    cv2.imwrite(f"debug_frames/frame_{fno}.jpg", annotated)

cap.release()
print("Selesai. Cek folder debug_frames.")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8x.p
t to 'yolov8x.pt': 100% ━━━━━━━━━━━━ 130.5MB 6.3MB/s 20.6s 20.5s<0.0s9

Frame 300
  person | conf=0.898
  sports ball | conf=0.377
  sports ball | conf=0.365
  sports ball | conf=0.312
  sports ball | conf=0.113
  skateboard | conf=0.055

Frame 800
  sports ball | conf=0.846
  person | conf=0.846
  handbag | conf=0.222
  handbag | conf=0.188
  cell phone | conf=0.086

Frame 1200
  person | conf=0.910
  sports ball | conf=0.903
  sports ball | conf=0.806
  sports ball | conf=0.087

Frame 1800
  person | conf=0.909
  sports ball | conf=0.866
  sports ball | conf=0.841
  sports ball | conf=0.211

Frame 2500
  person | conf=0.886
  sports ball | conf=0.779
  sports ball | conf=0.734
  sports ball | conf=0.056

Frame 3200
  person | conf=0.858
  sports ball | conf=0.829
  sports ball | conf=0.544

Frame 4000
  person | conf=0.881
  sports ball | conf=0.873
  sports ball | conf=0.832

Frame 5000
  person | conf=0.857
  sports ball | conf=0.557
  sports ball | conf=0.416
  sports ball | conf=0.252
Selesai. Cek folder debug_frames.
```

Soal 1: Ekstraksi & YOLO Tracking (15%)

1.1 Deskripsi

Pada bagian ini, kami mengimplementasikan **YOLO Tracking** menggunakan `model.track()` dari library Ultralytics untuk mendeteksi dan melacak ketiga objek bola basket secara simultan pada video 3 menit.

1.2 Penjelasan Metode

YOLOv8 + Object Tracking

YOLO (You Only Look Once) adalah arsitektur deteksi objek real-time yang memproses seluruh gambar dalam satu forward pass. YOLOv8 merupakan versi terbaru dari keluarga YOLO yang dikembangkan oleh Ultralytics.

Pipeline tracking yang digunakan:

1. **Detection:** YOLOv8 mendeteksi objek pada setiap frame dan menghasilkan bounding box beserta class prediction
2. **Tracking:** Algoritma **BoT-SORT** mencocokkan deteksi antar frame untuk memberikan **ID unik** yang konsisten pada setiap objek
3. **Filtering:** Hanya objek dengan class `sports ball` (class ID 32 pada COCO dataset) yang diambil

Parameter Kunci

Parameter	Nilai	Alasan
Model	`yolov8x.pt`	Extra-large — akurasi lebih tinggi untuk objek kecil seperti bola basket
Confidence threshold	0.05	Cukup rendah untuk menangkap bola yang blur atau kecil
IOU threshold	0.4	Mengurangi duplikasi deteksi pada objek kecil
Image size	1280	Resolusi tinggi meningkatkan sensitivitas terhadap small object
Tracker	BoT-SORT (default)	Lebih robust untuk multi-object tracking

Mengapa YOLOv8?

- **Real-time performance:** Mampu memproses frame video dengan efisien
- **Built-in tracking:** Method `model.track()` sudah mengintegrasikan algoritma multi-object tracking secara langsung
- **Pre-trained pada COCO:** Sudah mengenali 80 class objek termasuk `sports ball`
- **Persistent ID:** Parameter `persist=True` menjaga ID bola tetap konsisten antar frame

In [12]:
```
# SOAL 1: YOLO TRACKING — Deteksi & Lacak 3 Bola Basket

from ultralytics import YOLO
import cv2
import pandas as pd
import os
import time

model = YOLO("yolov8x.pt")
VIDEO_PATH = "output/video_3menit.mp4"
CSV_PATH = "output/ball_tracking_raw.csv"

if os.path.exists(CSV_PATH):
    df_tracking = pd.read_csv(CSV_PATH)
    print("Sudah ada CSV:", CSV_PATH)
else:
    cap = cv2.VideoCapture(VIDEO_PATH)
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    cap.release()

    tracking_data = []
    frame_idx = 0
    start_time = time.time()

    results = model.track(
        source=VIDEO_PATH,
        persist=True,
```

```
        stream=True,
        conf=0.05,
        imgsz=1280,
        iou=0.4,
        verbose=False
    )

    for result in results:
        frame_idx += 1

        if result.boxes is None or len(result.boxes) == 0:
            continue

        boxes = result.boxes
        for i in range(len(boxes)):
            cls_id = int(boxes.cls[i].item())
            cls_name = model.names[cls_id]
            conf = float(boxes.conf[i].item())

            if cls_name != "sports ball":
                continue

            x1, y1, x2, y2 = boxes.xyxy[i].cpu().numpy()
            track_id = int(boxes.id[i].item()) if boxes.id is not None else -1

            tracking_data.append({
                "frame": frame_idx,
                "track_id": track_id,
                "center_x": (x1 + x2) / 2,
                "center_y": (y1 + y2) / 2,
                "bbox_x1": float(x1),
                "bbox_y1": float(y1),
                "bbox_x2": float(x2),
                "bbox_y2": float(y2),
                "confidence": conf
            })

        if frame_idx % 100 == 0:
            elapsed = time.time() - start_time
            print(f"frame {frame_idx}/{total_frames}, deteksi {len(tracking_data)},

    df_tracking = pd.DataFrame(tracking_data)
    df_tracking.to_csv(CSV_PATH, index=False)
    print("Selesai:", CSV_PATH)
```

```
frame 100/5400, deteksi 142, 0.6 FPS
frame 200/5400, deteksi 299, 0.6 FPS
frame 300/5400, deteksi 459, 0.6 FPS
frame 400/5400, deteksi 619, 0.6 FPS
frame 500/5400, deteksi 770, 0.6 FPS
frame 600/5400, deteksi 941, 0.6 FPS
frame 700/5400, deteksi 1098, 0.6 FPS
frame 800/5400, deteksi 1265, 0.6 FPS
frame 900/5400, deteksi 1422, 0.6 FPS
frame 1000/5400, deteksi 1569, 0.6 FPS
frame 1100/5400, deteksi 1736, 0.6 FPS
frame 1200/5400, deteksi 1889, 0.6 FPS
frame 1300/5400, deteksi 2038, 0.6 FPS
frame 1400/5400, deteksi 2187, 0.6 FPS
frame 1500/5400, deteksi 2352, 0.6 FPS
frame 1600/5400, deteksi 2511, 0.6 FPS
frame 1700/5400, deteksi 2655, 0.6 FPS
frame 1800/5400, deteksi 2804, 0.6 FPS
frame 1900/5400, deteksi 2977, 0.6 FPS
frame 2000/5400, deteksi 3144, 0.6 FPS
frame 2100/5400, deteksi 3309, 0.6 FPS
frame 2200/5400, deteksi 3452, 0.6 FPS
frame 2300/5400, deteksi 3626, 0.6 FPS
frame 2400/5400, deteksi 3769, 0.6 FPS
frame 2500/5400, deteksi 3920, 0.6 FPS
frame 2600/5400, deteksi 4063, 0.6 FPS
frame 2700/5400, deteksi 4213, 0.6 FPS
frame 2800/5400, deteksi 4385, 0.6 FPS
frame 2900/5400, deteksi 4536, 0.6 FPS
frame 3000/5400, deteksi 4686, 0.6 FPS
frame 3100/5400, deteksi 4842, 0.6 FPS
frame 3200/5400, deteksi 5025, 0.6 FPS
frame 3300/5400, deteksi 5181, 0.6 FPS
frame 3400/5400, deteksi 5350, 0.6 FPS
frame 3500/5400, deteksi 5504, 0.6 FPS
frame 3600/5400, deteksi 5669, 0.6 FPS
frame 3700/5400, deteksi 5800, 0.6 FPS
frame 3800/5400, deteksi 5949, 0.6 FPS
frame 3900/5400, deteksi 6098, 0.6 FPS
frame 4000/5400, deteksi 6264, 0.6 FPS
frame 4100/5400, deteksi 6424, 0.6 FPS
frame 4200/5400, deteksi 6586, 0.6 FPS
frame 4300/5400, deteksi 6716, 0.6 FPS
frame 4400/5400, deteksi 6870, 0.6 FPS
frame 4500/5400, deteksi 7022, 0.6 FPS
frame 4600/5400, deteksi 7201, 0.6 FPS
frame 4700/5400, deteksi 7362, 0.6 FPS
frame 4800/5400, deteksi 7536, 0.6 FPS
frame 4900/5400, deteksi 7693, 0.6 FPS
frame 5000/5400, deteksi 7862, 0.6 FPS
frame 5100/5400, deteksi 7999, 0.6 FPS
frame 5200/5400, deteksi 8163, 0.6 FPS
frame 5300/5400, deteksi 8318, 0.6 FPS
frame 5400/5400, deteksi 8466, 0.6 FPS
Selesai: output/ball_tracking_raw.csv
```

In [5]:

```python
# 1.2 Analisis Hasil Tracking

import pandas as pd

CSV_PATH = "output/ball_tracking_raw.csv"
df_tracking = pd.read_csv(CSV_PATH)

print("=" * 60)
print("RINGKASAN HASIL TRACKING")
print("=" * 60)

print(f"\nTotal deteksi        : {len(df_tracking):,}")
print(f"Jumlah frame unik    : {df_tracking['frame'].nunique():,}")
print(f"Jumlah track ID unik : {df_tracking['track_id'].nunique()}")

track_stats = df_tracking.groupby("track_id").agg(
    jumlah_frame=("frame", "count"),
```

```
    frame_awal=("frame", "min"),
    frame_akhir=("frame", "max"),
    avg_conf=("confidence", "mean"),
    avg_x=("center_x", "mean"),
    avg_y=("center_y", "mean")
).round(2)

print("\nTrack ID yang terdeteksi:")
print(track_stats.to_string())

print("\n5 baris pertama data tracking:")
print(df_tracking.head().to_string())
```

```
============================================================
RINGKASAN HASIL TRACKING
============================================================

Total deteksi        : 8,466
Jumlah frame unik    : 5,005
Jumlah track ID unik : 849

Track ID yang terdeteksi:
          jumlah_frame  frame_awal  frame_akhir  avg_conf   avg_x   avg_y
track_id
2                   11           1           44      0.77  625.51  309.14
3                   17           1           24      0.64  677.43  242.97
7                    3           4            6      0.55  747.20  168.77
14                   2          10           11      0.80  640.83  363.55
18                  12          16           34      0.77  560.46  235.52
19                   5          18           73      0.77  739.81  513.12
21                   3          27           31      0.67  690.94  283.72
26                   6          33           38      0.56  760.19  252.24
32                  14          42           59      0.77  564.53  254.21
36                  11          50           88      0.62  688.90  296.33
37                   8          52           73      0.80  641.37  271.06
43                  25          58          285      0.75  605.71  374.61
44                   6          58          111      0.49  749.59  168.51
49                  10          63           72      0.82  636.38  279.17
53                   7          66           72      0.80  545.16  170.86
57                  19          71          333      0.75  738.77  475.67
59                  10          75           85      0.74  583.45  347.13
65                   8          88          200      0.62  754.42  251.67
67                  10          90          102      0.71  668.23  267.44
69                   4          92          118      0.80  754.97  290.77
71                  14          96          109      0.82  568.13  272.87
78                   3         106          131      0.67  692.77  325.33
79                   4         109          113      0.55  663.12  264.18
81                   8         114          122      0.80  634.86  330.50
82                   6         120          125      0.72  552.20  168.46
88                   9         128          136      0.76  573.88  335.31
89                   7         134          140      0.65  659.19  267.67
90                   3         135          137      0.52  745.28  169.16
97                  10         143          152      0.71  633.32  269.08
103                 16         147          165      0.81  558.18  263.20
110                 10         162          184      0.64  591.59  252.44
112                  8         164          248      0.73  605.38  503.70
113                  5         164          168      0.56  750.05  192.33
116                  9         170          178      0.77  628.96  270.02
119                  7         173          179      0.80  540.82  185.67
126                 23         185          211      0.75  581.45  227.64
134                  9         196          204      0.74  633.75  291.62
144                  1         210          210      0.58  671.04  202.53
145                  6         214          220      0.77  664.80  276.37
149                  4         217          220      0.82  738.70  170.42
150                  5         219          273      0.77  612.27  514.98
156                 11         224          258      0.75  636.98  302.28
160                 10         227          236      0.82  543.80  175.17
163                 13         230          248      0.59  739.28  196.79
164                 26         232          265      0.75  616.53  272.47
175                  2         258          259      0.90  730.32  525.85
178                  7         266          272      0.56  668.36  284.21
180                 10         270          279      0.66  764.29  207.69
181                 12         272          338      0.74  625.96  383.29
184                 14         281          294      0.79  547.28  212.93
190                  8         294          302      0.65  662.41  268.83
194                  3         300          302      0.51  763.77  189.21
199                 10         305          314      0.75  635.67  255.93
203                 19         308          326      0.80  558.49  276.05
206                  2         312          313      0.77  736.54  526.76
210                 11         319          356      0.62  680.75  287.86
211                  6         321          328      0.65  656.21  267.22
215                  3         327          329      0.63  749.85  189.84
220                 15         336          354      0.76  558.81  248.81
222                  3         339          341      0.79  728.37  509.93
227                 13         353          443      0.74  617.56  394.55
231                  2         355          356      0.52  764.33  207.71
```

236	13	359	371	0.60	670.48	239.82
238	9	359	501	0.73	748.26	354.37
240	16	364	379	0.81	559.82	291.33
247	8	373	381	0.69	663.64	280.15
248	6	377	382	0.58	752.26	188.48
251	12	385	396	0.70	615.87	259.73
255	15	389	403	0.80	548.31	236.64
261	6	399	408	0.53	659.62	286.45
263	4	406	434	0.86	601.36	540.65
266	12	410	422	0.72	625.86	279.58
267	6	410	436	0.66	755.30	190.98
268	4	412	466	0.70	752.93	270.48
269	16	415	435	0.78	548.07	227.42
274	19	421	649	0.71	716.53	439.55
277	5	428	432	0.69	673.76	283.49
282	18	443	460	0.79	551.40	271.69
287	20	453	502	0.69	656.52	280.03
288	11	456	481	0.72	603.50	253.52
295	8	468	478	0.71	630.58	246.96
297	8	469	476	0.83	545.13	172.16
309	8	488	502	0.66	750.35	265.23
311	4	491	517	0.79	592.88	445.46
317	14	498	511	0.76	546.06	223.66
320	27	508	543	0.72	596.05	260.60
322	12	513	530	0.81	747.13	242.10
323	26	515	717	0.77	616.66	426.20
327	7	521	528	0.76	621.70	281.27
333	7	536	544	0.70	682.17	279.90
335	7	542	549	0.66	752.19	211.36
337	11	546	556	0.77	638.29	289.25
343	17	552	570	0.74	561.64	257.63
349	7	564	570	0.70	680.64	274.99
351	3	568	572	0.59	761.67	184.51
354	2	573	592	0.81	610.07	412.11
355	22	575	599	0.70	707.06	216.92
358	10	576	715	0.77	752.92	353.00
361	10	579	588	0.77	554.96	193.81
367	20	587	615	0.67	610.17	246.72
376	10	601	611	0.71	638.07	269.44
388	27	614	642	0.67	597.05	237.87
393	5	620	625	0.40	747.99	185.40
400	8	629	637	0.80	636.64	264.56
405	6	636	689	0.77	740.74	518.70
413	6	646	651	0.62	745.80	183.05
418	8	654	663	0.74	652.18	285.86
421	14	660	673	0.78	559.37	253.66
428	8	670	677	0.67	681.08	286.34
429	7	673	681	0.72	763.09	221.43
430	10	676	839	0.61	611.03	538.74
436	9	681	690	0.66	656.05	282.58
439	13	687	703	0.73	562.11	261.40
444	7	697	704	0.66	680.96	276.87
446	6	702	707	0.59	764.50	208.96
450	17	713	730	0.78	566.84	281.01
455	10	723	733	0.53	661.10	279.22
456	7	729	736	0.61	748.39	208.31
458	11	733	744	0.77	630.53	293.10
461	17	739	757	0.80	561.92	264.22
464	4	743	771	0.75	735.99	512.90
467	5	750	755	0.51	698.47	301.14
468	4	754	784	0.48	749.68	182.65
473	6	760	789	0.59	758.25	249.12
475	8	762	773	0.69	642.94	280.14
476	3	763	772	0.78	739.69	353.33
478	14	768	782	0.76	569.79	285.76
484	8	778	786	0.64	672.25	270.41
486	18	787	881	0.76	620.83	314.72
487	10	789	798	0.75	632.86	265.99
494	13	795	812	0.81	560.83	241.02
495	9	795	804	0.61	714.10	408.32
497	8	807	814	0.59	655.10	269.78
498	7	808	838	0.60	745.11	174.64
502	9	813	853	0.70	753.59	270.67

505	8	816	823	0.80	642.26	279.84
506	2	816	817	0.75	750.61	290.08
511	14	821	840	0.79	556.95	231.40
514	12	823	840	0.65	678.11	331.52
521	6	843	848	0.78	638.18	301.17
523	13	847	859	0.79	545.81	213.38
525	13	851	1016	0.74	740.12	521.95
529	26	856	886	0.76	602.35	247.51
534	6	865	871	0.66	758.50	204.14
550	6	889	894	0.74	640.39	273.14
556	2	894	895	0.59	761.75	191.49
561	9	898	906	0.79	640.64	270.56
565	19	902	920	0.76	571.86	289.37
570	1	908	908	0.87	727.89	456.57
572	9	913	921	0.66	674.67	282.55
575	6	922	934	0.77	757.06	294.29
581	8	926	934	0.71	659.39	272.37
583	15	929	943	0.80	555.67	220.91
588	8	941	948	0.65	676.65	270.99
593	8	947	962	0.69	760.17	256.24
597	7	952	959	0.81	657.63	286.50
601	15	957	975	0.80	561.65	248.59
608	5	966	970	0.75	699.25	315.02
613	2	976	977	0.47	774.06	192.66
616	7	979	985	0.82	647.05	288.46
622	15	985	1002	0.73	566.74	269.62
630	9	994	1002	0.64	691.36	282.75
632	8	999	1007	0.75	766.26	198.41
634	3	1001	1028	0.67	628.03	546.82
637	12	1006	1019	0.71	656.84	266.91
641	14	1012	1025	0.74	567.19	247.54
643	26	1014	1292	0.77	735.46	504.61
645	7	1021	1027	0.69	692.33	289.13
648	4	1028	1031	0.54	762.09	181.14
652	19	1034	1058	0.61	699.57	221.25
653	19	1035	1101	0.71	671.42	283.48
656	15	1039	1053	0.81	559.51	253.89
659	21	1048	1075	0.68	603.88	231.41
665	9	1061	1070	0.77	638.83	274.98
671	6	1077	1082	0.81	594.22	427.17
673	2	1083	1085	0.81	755.78	194.24
679	10	1088	1097	0.82	656.00	276.60
682	8	1091	1098	0.77	546.97	178.84
686	8	1103	1110	0.75	689.07	284.61
698	2	1111	1112	0.39	758.71	202.25
703	9	1115	1123	0.72	668.63	273.56
708	15	1121	1136	0.80	562.96	284.75
711	11	1130	1140	0.61	671.28	276.82
714	9	1136	1150	0.55	757.53	264.41
718	8	1142	1150	0.77	633.68	273.87
722	10	1146	1155	0.73	544.13	180.77
727	7	1157	1163	0.81	583.81	418.63
728	9	1158	1166	0.62	672.06	276.23
730	9	1161	1171	0.69	750.22	221.87
735	7	1169	1176	0.80	636.21	285.33
737	18	1173	1190	0.82	558.67	277.55
743	11	1181	1191	0.79	681.80	305.75
744	8	1190	1221	0.64	736.66	176.96
749	11	1196	1206	0.76	629.12	262.83
750	5	1197	1204	0.73	741.52	380.43
752	17	1200	1219	0.81	556.24	247.40
758	18	1216	1236	0.68	587.65	219.00
767	16	1224	1245	0.70	690.69	222.03
768	6	1225	1307	0.71	751.65	279.63
777	8	1238	1245	0.78	584.22	399.20
778	7	1239	1246	0.66	683.10	291.39
786	6	1251	1256	0.74	643.18	295.07
788	1	1254	1254	0.79	745.42	369.24
789	14	1255	1268	0.82	553.75	214.78
795	5	1266	1272	0.51	676.23	290.16
804	8	1278	1286	0.70	625.26	277.03
807	18	1283	1300	0.81	561.66	280.10
814	5	1297	1319	0.60	605.09	259.08

815	6	1298	1303	0.77	748.79	179.19
823	9	1306	1314	0.72	628.33	267.72
825	11	1307	1317	0.75	549.89	180.01
828	12	1313	1327	0.73	684.68	366.33
840	8	1330	1341	0.71	750.28	311.79
844	8	1333	1340	0.83	639.93	281.88
845	18	1337	1354	0.80	559.94	298.49
847	8	1340	1422	0.81	734.66	529.10
853	23	1346	1373	0.76	600.91	236.31
862	8	1360	1367	0.77	625.02	269.54
863	2	1360	1361	0.70	751.01	307.85
870	6	1375	1436	0.79	583.98	374.77
872	6	1376	1383	0.63	670.24	275.83
878	5	1380	1384	0.65	748.42	177.57
879	18	1381	1598	0.79	590.53	436.23
883	12	1386	1397	0.74	650.16	264.70
886	11	1390	1400	0.79	545.04	187.75
894	6	1403	1411	0.58	678.75	282.21
895	9	1403	1411	0.61	751.86	177.72
903	17	1413	1437	0.66	694.29	235.19
908	15	1418	1432	0.81	561.55	246.11
913	30	1425	1487	0.76	597.69	256.48
917	9	1440	1449	0.77	638.36	278.26
922	2	1447	1448	0.81	738.46	530.95
926	23	1453	1482	0.73	591.23	234.18
929	2	1457	1458	0.48	723.65	170.02
935	11	1465	1475	0.80	645.09	285.92
936	2	1467	1468	0.66	751.79	290.19
940	9	1474	1485	0.75	705.57	380.38
946	9	1488	1519	0.66	645.83	269.05
948	2	1490	1491	0.80	748.74	186.55
949	8	1493	1500	0.79	630.34	296.79
951	3	1495	1550	0.55	745.68	278.35
952	11	1497	1508	0.76	541.47	190.53
957	9	1502	1511	0.74	696.79	406.84
965	8	1516	1602	0.74	606.41	507.50
973	8	1523	1530	0.72	627.58	252.17
977	12	1525	1536	0.76	544.28	194.78
988	21	1536	1560	0.71	612.32	232.70
993	7	1542	1548	0.62	742.19	180.29
998	11	1549	1559	0.74	633.65	256.57
1005	4	1557	1586	0.83	725.23	512.86
1007	10	1564	1573	0.58	675.73	277.96
1015	9	1573	1584	0.62	744.67	343.53
1018	8	1576	1583	0.79	641.25	290.65
1020	17	1581	1599	0.79	557.62	269.36
1025	23	1591	1618	0.71	606.47	234.96
1033	8	1604	1611	0.75	628.06	275.08
1041	5	1611	1667	0.83	728.88	519.73
1046	27	1617	1656	0.76	601.39	264.99
1053	5	1624	1628	0.57	746.44	183.27
1054	9	1625	1685	0.75	610.10	458.84
1057	9	1629	1638	0.77	633.37	295.06
1066	9	1645	1653	0.69	671.63	286.74
1069	7	1650	1659	0.65	745.13	223.25
1073	6	1658	1664	0.83	632.16	281.97
1076	16	1661	1680	0.78	553.26	240.68
1078	14	1665	1706	0.66	690.69	260.47
1086	17	1673	1750	0.72	609.83	234.36
1089	5	1676	1699	0.74	614.38	266.56
1092	16	1679	1923	0.81	595.46	493.49
1100	10	1688	1697	0.83	543.56	184.21
1118	4	1712	1720	0.77	733.74	354.36
1119	6	1712	1717	0.76	619.35	295.96
1120	18	1715	1732	0.80	555.55	249.07
1122	13	1719	1858	0.82	725.33	526.35
1126	4	1725	1754	0.73	695.44	352.54
1127	23	1728	1782	0.77	591.02	229.57
1128	9	1730	1754	0.73	607.32	254.03
1129	9	1731	1741	0.64	740.07	213.26
1134	5	1740	1744	0.75	622.57	289.27
1146	12	1762	1817	0.68	739.51	175.61
1151	7	1767	1775	0.69	635.66	275.79

1152	4	1767	1776	0.76	729.32	346.46
1154	1	1778	1778	0.80	692.11	416.00
1159	28	1785	1816	0.78	570.09	251.08
1168	8	1793	1800	0.84	636.25	288.94
1180	9	1808	1816	0.76	677.05	290.02
1185	3	1815	1844	0.73	605.71	532.73
1186	13	1819	1831	0.67	640.22	278.41
1189	4	1821	1829	0.71	738.86	365.89
1193	14	1826	1839	0.80	550.34	217.76
1196	26	1834	1863	0.70	598.36	238.40
1197	6	1841	1872	0.55	743.18	175.03
1203	6	1849	1854	0.80	636.20	291.45
1204	6	1850	1911	0.73	735.84	370.19
1212	2	1860	1886	0.60	704.30	405.88
1214	7	1863	1897	0.63	678.80	293.87
1216	8	1866	1873	0.54	629.63	282.06
1219	3	1870	1897	0.55	603.44	546.03
1223	10	1875	1884	0.76	621.88	281.24
1224	17	1877	1899	0.77	549.42	231.93
1226	42	1882	2051	0.73	644.00	347.26
1233	9	1896	1904	0.66	742.04	227.16
1235	8	1901	1908	0.76	638.94	309.74
1238	17	1906	1924	0.77	555.92	264.03
1245	19	1917	1938	0.62	620.62	242.32
1247	11	1922	1938	0.64	740.98	254.14
1249	29	1926	2231	0.78	606.88	402.24
1252	7	1930	1937	0.75	630.01	272.63
1254	9	1940	1949	0.83	571.21	318.60
1255	13	1941	1964	0.64	629.93	258.47
1262	10	1955	1964	0.79	631.13	279.38
1272	11	1967	1977	0.80	576.37	351.76
1275	9	1970	1980	0.63	673.35	287.54
1277	6	1974	1979	0.53	730.92	177.00
1283	5	1983	1987	0.81	638.29	304.34
1286	18	1986	2005	0.76	565.33	262.60
1293	10	1997	2007	0.65	667.45	285.78
1295	3	2000	2002	0.59	734.06	172.36
1299	5	2010	2048	0.80	736.41	318.70
1300	7	2010	2016	0.75	638.76	278.42
1305	16	2014	2031	0.83	558.76	268.73
1307	9	2017	2099	0.84	728.53	514.81
1311	3	2031	2033	0.53	745.98	195.70
1315	13	2036	2055	0.68	650.82	241.21
1320	10	2051	2060	0.72	663.19	282.77
1330	1	2061	2061	0.73	747.39	219.75
1333	7	2063	2070	0.77	639.82	280.48
1334	4	2063	2072	0.81	733.27	346.39
1336	10	2067	2076	0.80	543.91	195.15
1345	6	2080	2085	0.75	638.16	275.08
1347	6	2081	2086	0.66	740.18	173.38
1350	10	2087	2096	0.83	634.96	315.11
1351	2	2091	2092	0.89	742.91	319.74
1354	16	2094	2111	0.84	557.05	269.13
1356	5	2097	2155	0.63	726.88	502.02
1358	20	2103	2130	0.70	600.37	235.74
1359	6	2108	2116	0.65	751.67	195.72
1363	9	2116	2124	0.68	638.94	272.60
1369	8	2130	2137	0.64	687.28	295.53
1376	9	2137	2146	0.72	747.20	238.79
1383	11	2143	2156	0.75	659.38	258.35
1385	9	2149	2157	0.81	546.84	196.90
1388	8	2151	2235	0.80	726.34	498.77
1394	6	2160	2166	0.53	675.04	272.05
1402	2	2167	2195	0.51	747.71	213.63
1405	11	2170	2180	0.68	619.75	263.25
1410	10	2175	2184	0.81	543.58	193.08
1416	25	2185	2212	0.72	593.94	229.48
1421	7	2191	2220	0.60	743.11	172.29
1426	6	2195	2249	0.79	600.35	426.06
1427	10	2197	2207	0.72	647.90	265.91
1433	8	2205	2286	0.88	726.35	519.88
1435	24	2211	2239	0.68	605.21	239.59
1445	5	2227	2307	0.61	745.71	322.90

1451	25	2238	2266	0.77	606.28	235.30
1459	2	2250	2251	0.71	748.88	255.59
1460	12	2251	2263	0.77	636.49	257.24
1467	10	2265	2275	0.68	664.11	287.02
1471	4	2270	2273	0.65	739.67	167.44
1474	4	2273	2326	0.81	604.54	545.18
1477	12	2277	2289	0.70	626.75	275.35
1478	8	2279	2441	0.69	746.65	298.14
1480	17	2282	2298	0.80	553.64	252.07
1484	21	2290	2318	0.76	598.50	243.44
1491	11	2304	2316	0.71	637.93	289.10
1494	10	2313	2421	0.78	723.80	499.90
1498	6	2320	2325	0.80	579.77	388.95
1500	22	2321	2346	0.72	592.83	227.85
1501	6	2324	2329	0.70	746.96	182.36
1504	9	2331	2339	0.78	636.23	296.34
1510	8	2347	2356	0.64	662.45	292.98
1511	16	2348	2419	0.80	615.06	318.96
1515	27	2353	2609	0.76	613.92	446.52
1519	7	2358	2364	0.80	639.78	298.59
1522	11	2362	2372	0.80	547.47	201.12
1529	7	2373	2380	0.72	664.25	294.16
1538	8	2384	2391	0.72	643.99	293.48
1540	12	2388	2400	0.76	549.11	211.28
1547	17	2400	2424	0.67	596.74	226.16
1552	9	2404	2412	0.61	754.91	216.04
1572	8	2429	2452	0.71	614.46	270.26
1574	11	2430	2447	0.68	746.65	239.03
1577	4	2436	2490	0.78	598.56	397.94
1578	10	2438	2447	0.76	631.50	263.82
1581	8	2442	2449	0.72	539.33	185.08
1583	6	2446	2501	0.79	726.77	506.41
1588	26	2454	2507	0.70	596.01	245.26
1593	2	2459	2485	0.43	745.06	173.42
1597	8	2465	2472	0.77	639.18	275.12
1598	2	2466	2467	0.76	744.01	317.64
1601	14	2469	2488	0.81	551.30	237.11
1612	8	2492	2499	0.63	624.87	278.45
1616	2	2499	2500	0.93	716.74	525.67
1620	18	2506	2530	0.66	591.66	233.33
1632	12	2518	2529	0.70	630.55	268.54
1633	2	2518	2519	0.72	741.51	281.41
1637	7	2526	2609	0.79	723.18	502.27
1641	16	2532	2596	0.72	642.33	249.49
1642	8	2533	2541	0.63	674.10	290.49
1643	6	2534	2539	0.82	586.13	424.47
1644	9	2536	2546	0.66	744.64	211.65
1649	5	2546	2550	0.77	642.65	298.10
1652	8	2550	2557	0.82	545.78	186.90
1654	4	2553	2607	0.85	723.11	539.33
1658	7	2558	2656	0.66	717.25	365.70
1659	24	2560	2586	0.71	590.26	237.61
1670	7	2573	2652	0.70	751.91	218.72
1682	8	2590	2613	0.78	613.20	266.89
1686	3	2594	2622	0.73	606.09	527.21
1695	10	2602	2611	0.78	539.20	180.15
1700	6	2615	2621	0.50	673.91	285.13
1711	7	2627	2633	0.77	635.28	286.09
1713	6	2627	2737	0.71	745.67	287.98
1715	17	2631	2648	0.84	551.25	282.65
1718	22	2635	2760	0.71	687.56	388.41
1720	1	2637	2637	0.62	669.51	218.18
1725	22	2643	2664	0.69	565.66	233.49
1730	11	2653	2663	0.83	606.31	276.00
1738	10	2666	2703	0.80	563.01	300.35
1740	15	2667	2690	0.69	603.06	239.63
1741	7	2669	2675	0.83	585.49	423.01
1749	6	2681	2686	0.74	641.37	292.47
1754	7	2696	2703	0.55	686.73	283.78
1755	2	2698	2703	0.58	742.14	180.05
1758	12	2705	2716	0.84	638.63	292.91
1761	19	2711	2730	0.78	561.19	268.54
1762	30	2715	2803	0.75	613.02	284.70

1764	9	2721	2732	0.58	669.07	280.25
1767	9	2726	2736	0.70	746.43	200.95
1775	8	2735	2742	0.81	638.30	281.18
1777	19	2738	2756	0.82	556.11	267.92
1783	3	2756	2758	0.68	749.16	173.28
1789	11	2762	2772	0.77	630.82	253.37
1792	16	2764	2786	0.69	553.66	224.69
1802	2	2782	2783	0.50	737.09	169.46
1808	6	2789	2794	0.79	641.84	294.38
1809	2	2790	2791	0.77	743.05	319.91
1812	13	2796	2808	0.76	697.45	376.53
1821	16	2810	2904	0.74	613.66	363.74
1824	2	2812	2813	0.62	759.66	193.67
1826	18	2813	2831	0.74	560.00	216.17
1827	12	2814	2825	0.83	650.88	285.00
1829	7	2817	2903	0.79	743.72	344.38
1839	8	2833	2841	0.58	661.79	272.14
1844	4	2837	2840	0.58	756.10	194.05
1848	9	2843	2851	0.71	624.81	270.68
1853	12	2849	2864	0.76	558.23	269.70
1860	7	2857	2865	0.49	663.43	282.76
1861	3	2861	2865	0.51	738.73	188.31
1863	5	2863	2894	0.75	607.50	480.13
1867	11	2869	2879	0.74	623.55	266.22
1869	17	2873	2891	0.81	556.56	263.96
1878	10	2883	2893	0.68	670.14	279.23
1881	2	2889	2890	0.41	732.37	170.66
1883	4	2896	2925	0.68	744.87	285.04
1889	12	2900	2911	0.73	541.96	196.90
1895	22	2909	2963	0.70	613.78	247.88
1896	8	2911	2922	0.53	664.29	280.41
1902	12	2918	2983	0.81	617.57	372.46
1903	2	2919	2920	0.51	748.62	194.23
1907	7	2923	2929	0.75	627.23	288.31
1910	15	2928	2942	0.81	550.00	260.21
1914	4	2931	2959	0.88	714.24	526.58
1923	9	2950	2958	0.77	636.25	281.65
1924	4	2950	2972	0.76	692.63	360.41
1934	19	2967	2990	0.80	568.96	222.50
1938	8	2971	2984	0.61	753.63	290.61
1950	9	2991	2999	0.69	683.93	281.38
1951	7	2992	2998	0.75	593.21	441.96
1952	5	2995	3000	0.47	746.94	178.95
1958	9	3004	3013	0.69	647.63	265.12
1961	14	3008	3026	0.82	557.53	244.80
1973	9	3019	3027	0.66	660.85	280.19
1984	13	3028	3092	0.72	640.71	316.77
1986	6	3030	3035	0.86	650.37	295.56
1988	11	3034	3044	0.77	545.01	193.87
1992	4	3038	3065	0.86	726.10	524.29
1997	14	3043	3081	0.72	659.57	300.37
1998	24	3045	3072	0.74	584.48	232.42
2005	14	3051	3169	0.71	626.32	393.37
2006	11	3051	3163	0.60	747.87	202.19
2009	1	3054	3054	0.60	760.91	205.32
2010	6	3057	3062	0.71	644.09	282.32
2011	3	3057	3067	0.71	734.18	337.88
2023	1	3079	3079	0.87	152.82	594.50
2030	15	3083	3123	0.73	550.75	209.62
2033	50	3085	3135	0.80	212.37	565.30
2034	16	3086	3101	0.85	548.44	244.13
2043	4	3102	3105	0.57	658.91	278.45
2044	6	3104	3158	0.83	603.91	527.81
2048	8	3109	3116	0.75	638.90	291.74
2049	5	3109	3113	0.64	747.57	339.99
2061	3	3121	3148	0.59	698.01	378.00
2062	5	3123	3130	0.60	729.06	183.87
2065	9	3126	3148	0.74	606.55	263.10
2070	10	3135	3144	0.76	634.31	277.53
2074	7	3139	3145	0.84	534.35	175.88
2080	5	3142	3155	0.66	699.80	421.20
2098	4	3162	3215	0.72	744.31	294.93
2100	17	3166	3183	0.82	554.86	276.97

2103	8	3169	3178	0.65	697.48	392.30
2110	18	3182	3202	0.62	582.26	219.70
2112	10	3185	3302	0.69	743.93	297.71
2114	6	3188	3194	0.65	623.46	288.01
2120	30	3196	3229	0.72	599.69	266.62
2133	3	3213	3232	0.78	588.96	394.11
2134	7	3213	3277	0.64	733.19	368.00
2136	10	3216	3225	0.79	631.87	245.39
2140	10	3221	3356	0.80	722.86	513.80
2142	7	3228	3236	0.59	673.79	298.20
2151	9	3241	3251	0.68	642.05	266.62
2153	17	3246	3262	0.81	557.23	300.31
2157	30	3252	3315	0.78	590.09	272.23
2160	3	3262	3265	0.62	753.35	193.85
2167	9	3268	3276	0.72	635.54	255.98
2178	20	3287	3308	0.72	575.44	222.42
2182	4	3289	3292	0.78	748.13	194.49
2186	7	3295	3303	0.76	650.78	274.41
2188	5	3296	3375	0.76	744.61	318.65
2193	21	3309	3335	0.77	580.31	235.37
2199	6	3315	3321	0.70	756.59	218.35
2204	10	3319	3328	0.77	633.81	298.67
2208	10	3328	3438	0.82	725.97	514.07
2210	23	3334	3363	0.73	588.85	239.64
2215	3	3340	3368	0.48	739.27	176.32
2220	15	3345	3396	0.58	703.22	288.75
2224	11	3347	3359	0.76	637.29	268.13
2229	22	3362	3386	0.66	589.70	232.26
2234	13	3368	3380	0.79	611.64	361.61
2238	3	3382	3384	0.59	639.45	232.82
2241	7	3388	3394	0.78	589.77	409.83
2242	7	3392	3400	0.65	743.54	202.71
2248	10	3400	3410	0.72	664.00	265.62
2252	16	3404	3422	0.81	556.97	252.24
2256	21	3414	3441	0.77	601.35	233.44
2257	7	3418	3424	0.64	758.22	180.10
2263	7	3427	3433	0.78	640.16	283.34
2269	31	3434	3468	0.75	595.76	271.76
2276	3	3445	3447	0.55	745.07	173.85
2279	11	3448	3585	0.73	604.08	506.95
2285	9	3454	3463	0.71	621.60	265.33
2290	9	3460	3470	0.64	708.39	393.56
2295	11	3472	3495	0.66	618.88	263.02
2305	6	3481	3486	0.75	646.31	281.46
2306	7	3481	3589	0.62	749.73	294.11
2308	7	3485	3491	0.85	538.21	177.58
2312	4	3488	3516	0.86	726.63	529.42
2316	27	3494	3521	0.65	602.64	239.94
2322	6	3500	3505	0.66	755.71	185.29
2327	8	3507	3514	0.75	644.53	281.92
2339	20	3525	3548	0.76	571.15	220.99
2342	4	3527	3531	0.55	754.89	186.87
2346	8	3532	3540	0.82	626.66	316.49
2352	7	3541	3547	0.70	706.90	443.51
2360	5	3553	3564	0.67	640.67	287.82
2361	16	3555	3648	0.73	634.83	399.82
2364	7	3558	3588	0.55	758.19	233.39
2366	3	3560	3567	0.76	633.84	311.55
2368	17	3562	3583	0.81	551.04	251.00
2374	7	3568	3575	0.74	704.50	438.51
2376	5	3577	3581	0.63	740.43	172.41
2377	24	3578	3602	0.72	580.78	232.60
2380	7	3587	3593	0.80	636.15	295.50
2397	2	3606	3607	0.77	660.22	277.54
2402	5	3611	3637	0.63	763.15	194.94
2406	11	3614	3624	0.69	663.56	264.38
2407	6	3614	3693	0.76	757.11	276.00
2411	10	3617	3626	0.82	547.40	200.95
2414	8	3621	3729	0.74	734.53	521.48
2434	17	3642	3662	0.75	560.22	253.04
2441	4	3654	3660	0.64	693.42	306.97
2443	6	3659	3664	0.52	756.12	198.08
2453	8	3669	3678	0.65	672.98	239.69

2456	17	3671	3687	0.81	566.80	292.47
2463	24	3681	3708	0.65	599.28	245.76
2466	4	3689	3745	0.61	760.44	209.20
2473	6	3694	3700	0.58	633.89	270.46
2475	2	3699	3700	0.81	742.47	531.71
2479	8	3708	3717	0.71	662.07	289.66
2485	12	3714	3725	0.76	640.64	372.23
2489	18	3724	3741	0.83	563.71	285.29
2491	11	3727	3744	0.64	729.20	197.93
2492	12	3728	3862	0.82	739.21	535.40
2495	6	3736	3742	0.71	672.85	289.42
2500	7	3745	3751	0.78	646.45	331.80
2501	2	3747	3748	0.77	760.82	273.99
2506	17	3751	3768	0.83	564.46	273.57
2518	10	3761	3770	0.77	665.32	291.56
2520	6	3766	3771	0.54	758.85	185.29
2527	8	3774	3783	0.73	649.01	266.31
2528	2	3774	3775	0.77	752.99	289.20
2530	14	3777	3790	0.82	555.52	221.88
2542	9	3792	3813	0.74	602.00	239.42
2550	2	3798	3799	0.60	762.60	214.51
2552	7	3800	3807	0.71	651.10	282.68
2557	6	3804	3809	0.75	548.01	171.37
2565	27	3815	3850	0.73	597.65	256.97
2574	16	3822	3888	0.72	644.97	371.29
2579	9	3828	3836	0.73	666.88	259.39
2580	2	3828	3829	0.73	764.08	295.19
2587	8	3843	3850	0.64	681.94	279.78
2591	5	3847	3851	0.55	763.45	181.58
2592	2	3852	3871	0.79	616.34	403.40
2593	10	3854	3864	0.74	663.71	274.43
2594	11	3855	4043	0.68	754.96	292.42
2595	12	3859	3870	0.78	561.28	229.64
2600	9	3867	3875	0.67	678.64	312.46
2601	6	3875	3931	0.71	616.11	494.48
2608	14	3885	3898	0.81	554.03	243.42
2611	2	3888	3889	0.78	735.66	529.54
2614	8	3895	3902	0.79	679.09	295.67
2616	5	3901	3905	0.58	757.82	178.69
2621	9	3907	3915	0.62	648.08	270.24
2624	16	3911	3929	0.80	562.94	253.02
2627	7	3915	3945	0.62	685.66	314.32
2632	9	3922	3930	0.54	668.33	281.61
2635	6	3927	3932	0.51	755.33	177.25
2639	5	3934	3938	0.67	641.61	301.46
2642	15	3937	3956	0.82	561.23	243.14
2646	12	3942	3953	0.71	692.57	380.01
2652	5	3954	3958	0.66	759.18	190.89
2656	12	3958	3994	0.76	630.06	311.51
2658	13	3960	3980	0.70	674.12	242.57
2660	18	3963	3980	0.83	558.79	281.89
2662	18	3968	4119	0.62	751.04	272.98
2667	8	3975	3983	0.72	655.67	287.15
2676	16	3990	4009	0.77	557.55	245.04
2683	17	4001	4027	0.82	593.52	216.25
2690	12	4012	4023	0.75	632.31	276.61
2691	10	4012	4150	0.69	767.14	226.82
2703	12	4029	4053	0.73	617.93	258.94
2709	14	4035	4154	0.76	626.41	404.85
2710	4	4035	4063	0.58	756.80	169.12
2714	7	4040	4046	0.82	642.50	286.39
2715	7	4041	4103	0.82	747.91	397.74
2716	7	4043	4049	0.81	547.40	177.94
2718	31	4048	4081	0.78	612.19	258.81
2720	1	4050	4050	0.42	669.98	216.11
2722	6	4055	4060	0.85	581.05	384.66
2725	11	4067	4077	0.77	639.73	274.72
2730	9	4075	4102	0.81	664.96	329.13
2732	10	4081	4090	0.76	672.44	286.32
2739	3	4089	4116	0.77	617.76	544.51
2743	12	4094	4130	0.79	654.10	297.10
2749	17	4098	4117	0.80	560.73	249.06
2754	8	4110	4118	0.56	676.39	275.50

2758	17	4126	4143	0.81	565.96	274.18
2760	1	4130	4130	0.83	728.74	535.20
2761	3	4132	4181	0.64	730.39	462.70
2767	5	4138	4143	0.64	668.01	279.69
2768	3	4143	4145	0.67	770.44	182.28
2771	4	4147	4172	0.77	611.32	438.98
2773	13	4153	4166	0.75	558.79	224.84
2779	10	4163	4172	0.69	684.77	279.84
2784	7	4173	4184	0.74	752.30	318.21
2787	10	4176	4188	0.69	666.28	261.84
2790	18	4180	4197	0.80	569.95	285.56
2795	10	4191	4200	0.66	671.57	278.53
2801	6	4200	4228	0.66	770.20	183.42
2803	7	4202	4209	0.64	648.11	288.81
2808	16	4209	4224	0.81	562.31	295.51
2814	12	4218	4282	0.64	662.61	291.15
2817	8	4222	4244	0.73	601.07	246.11
2824	9	4231	4239	0.74	626.22	269.71
2825	7	4231	4367	0.66	756.78	293.42
2828	7	4234	4240	0.83	539.62	172.45
2831	9	4238	4346	0.75	738.02	526.24
2843	8	4250	4272	0.77	600.85	257.36
2847	3	4253	4255	0.79	754.32	191.14
2849	14	4255	4269	0.80	657.23	276.34
2851	9	4260	4268	0.79	545.24	173.57
2857	2	4270	4271	0.69	696.45	359.07
2872	8	4282	4372	0.67	761.50	268.34
2874	6	4284	4289	0.74	643.91	292.31
2877	15	4288	4307	0.80	556.91	238.66
2880	2	4293	4319	0.56	673.48	226.93
2884	7	4300	4308	0.55	675.22	270.43
2891	8	4309	4316	0.81	646.60	318.89
2893	17	4316	4332	0.83	557.99	292.52
2898	24	4325	4352	0.71	599.67	231.57
2900	4	4330	4335	0.55	754.92	174.44
2904	9	4338	4346	0.78	622.90	269.78
2909	7	4345	4433	0.74	716.41	450.09
2912	8	4352	4359	0.58	681.47	285.20
2922	6	4360	4366	0.58	752.02	223.75
2927	18	4365	4390	0.67	700.82	222.37
2928	18	4369	4387	0.80	567.93	276.08
2935	6	4382	4387	0.64	681.40	269.35
2940	11	4392	4426	0.79	634.80	293.24
2941	5	4392	4446	0.63	754.90	286.64
2944	14	4395	4454	0.65	645.23	262.00
2946	14	4397	4414	0.76	565.91	254.51
2948	3	4399	4427	0.72	705.49	424.96
2951	7	4405	4411	0.70	681.75	300.81
2958	2	4415	4416	0.60	763.47	203.07
2963	18	4422	4439	0.82	564.19	283.39
2967	19	4426	4618	0.84	724.37	506.25
2972	4	4436	4440	0.62	742.37	182.21
2980	2	4445	4446	0.72	623.83	326.64
2982	17	4447	4467	0.80	551.30	244.07
2987	6	4460	4467	0.64	682.60	290.32
2993	7	4466	4548	0.72	593.12	500.29
2996	7	4468	4480	0.79	745.11	287.94
2998	9	4471	4479	0.71	635.74	283.95
3000	16	4475	4490	0.81	551.25	242.96
3006	29	4486	4521	0.75	575.26	260.10
3016	9	4498	4506	0.76	628.84	272.13
3027	23	4514	4539	0.72	590.08	233.99
3028	5	4515	4521	0.51	739.62	166.74
3029	14	4520	4665	0.76	623.67	401.26
3033	8	4525	4533	0.81	639.72	286.65
3035	2	4526	4527	0.79	742.60	290.61
3051	22	4544	4567	0.74	572.44	225.79
3053	2	4549	4550	0.65	761.74	197.03
3055	8	4552	4559	0.68	625.26	281.81
3056	6	4553	4615	0.73	740.81	339.85
3059	2	4561	4562	0.44	646.22	228.36
3062	25	4567	4593	0.65	600.12	239.82
3066	6	4571	4577	0.63	744.04	179.07

3071	9	4578	4587	0.82	634.49	296.52
3075	10	4587	4690	0.85	729.52	519.40
3077	22	4591	4618	0.78	613.50	255.40
3078	7	4595	4706	0.83	584.38	405.32
3079	12	4596	4607	0.58	740.49	212.74
3082	2	4600	4601	0.80	611.30	533.78
3085	10	4606	4617	0.68	654.96	260.91
3094	6	4622	4628	0.57	669.38	282.07
3100	2	4627	4628	0.67	753.55	188.61
3106	4	4632	4641	0.78	737.36	344.20
3107	11	4632	4643	0.59	667.84	263.22
3109	12	4637	4648	0.80	554.90	232.52
3113	9	4644	4652	0.66	675.23	312.02
3118	9	4653	4752	0.70	616.22	475.11
3126	3	4658	4667	0.81	736.22	352.21
3128	14	4662	4675	0.84	545.76	228.72
3131	10	4671	4680	0.68	670.33	290.04
3132	6	4677	4682	0.60	749.57	168.26
3134	9	4684	4692	0.77	629.92	284.98
3141	16	4689	4707	0.82	553.83	257.41
3148	26	4699	4728	0.72	585.63	236.47
3151	11	4709	4720	0.70	738.86	358.36
3154	13	4712	4725	0.68	658.81	255.48
3156	11	4722	4838	0.76	699.12	398.12
3159	11	4727	4762	0.74	658.70	296.69
3161	18	4730	4753	0.79	569.95	232.51
3174	17	4742	4762	0.64	722.63	214.91
3176	18	4747	5057	0.76	737.14	423.23
3187	19	4765	4785	0.60	694.65	223.44
3189	19	4767	4785	0.81	561.88	277.80
3192	26	4777	4831	0.71	605.94	254.27
3195	19	4783	4806	0.80	581.69	229.33
3198	2	4788	4789	0.67	763.05	206.11
3200	7	4791	4797	0.78	641.56	290.36
3215	6	4810	4815	0.51	752.94	179.27
3217	2	4812	4813	0.75	616.11	530.18
3219	6	4816	4821	0.80	639.99	331.01
3221	4	4818	4827	0.78	744.17	345.62
3224	3	4824	4826	0.60	671.07	237.03
3229	6	4833	4916	0.80	583.10	389.43
3231	5	4835	4842	0.40	757.00	185.21
3234	19	4838	4907	0.74	634.17	337.14
3238	6	4844	4849	0.76	640.97	285.49
3244	9	4849	4857	0.75	549.67	207.05
3250	7	4860	4867	0.57	665.81	275.12
3254	12	4863	4878	0.63	751.80	278.99
3259	8	4870	4878	0.75	645.47	279.85
3261	10	4876	4885	0.77	557.51	221.81
3263	20	4882	4908	0.70	621.43	243.66
3264	7	4889	4896	0.78	755.53	206.84
3275	13	4903	4917	0.68	679.05	348.02
3284	5	4915	4919	0.50	752.68	188.24
3289	8	4923	4930	0.73	639.27	260.64
3292	17	4925	4943	0.83	554.60	260.80
3294	29	4930	4964	0.80	615.13	269.55
3307	2	4947	4973	0.55	752.32	219.08
3308	7	4948	4954	0.84	627.87	299.41
3312	33	4956	4992	0.75	610.66	283.45
3313	2	4956	4958	0.48	653.00	239.61
3325	10	4975	4984	0.72	634.64	270.77
3332	30	4983	5014	0.67	616.40	282.48
3335	4	4994	4998	0.60	740.35	170.50
3340	8	4997	5103	0.85	603.72	539.60
3344	8	5002	5009	0.77	641.14	282.51
3348	27	5010	5043	0.78	615.31	281.91
3360	11	5027	5037	0.80	631.46	288.00
3362	2	5028	5029	0.78	744.94	262.48
3367	8	5036	5050	0.72	688.01	366.25
3382	9	5053	5061	0.71	633.80	309.47
3386	16	5059	5077	0.78	562.17	262.72
3388	2	5063	5064	0.79	720.26	518.54
3390	8	5069	5078	0.70	669.42	297.83
3400	6	5081	5086	0.81	626.85	301.09

```
3401                 8        5081         5143      0.81  738.68  402.71
3403                17        5084         5100      0.82  558.60  247.92
3413                10        5096         5131      0.70  677.34  291.06
3415                 7        5101         5107      0.61  749.89  194.00
3416                 2        5102         5103      0.51  667.88  274.53
3422                 9        5108         5118      0.69  642.49  268.98
3423                13        5112         5124      0.77  552.16  214.32
3435                 7        5126         5132      0.65  741.26  177.88
3438                 4        5129         5182      0.83  605.24  547.21
3442                11        5135         5146      0.66  623.62  259.41
3443                 1        5135         5135      0.60  611.00  206.26
3445                 3        5136         5144      0.84  731.21  373.39
3447                16        5140         5157      0.81  558.18  273.75
3452                18        5154         5176      0.69  575.54  222.03
3454                10        5160         5169      0.77  638.62  290.02
3455                 6        5160         5170      0.71  741.22  312.69
3467                 7        5176         5238      0.61  740.19  177.58
3470                 8        5178         5185      0.71  662.30  274.10
3474                13        5181         5196      0.68  750.58  266.88
3479                 8        5188         5195      0.74  638.78  287.83
3482                18        5192         5209      0.80  561.89  286.46
3490                 9        5203         5238      0.69  655.61  284.28
3491                10        5207         5229      0.69  591.77  247.82
3494                 2        5213         5232      0.80  602.42  399.17
3495                 9        5215         5223      0.80  639.51  277.01
3496                 2        5215         5216      0.63  749.82  272.65
3499                 7        5218         5224      0.86  540.93  177.11
3503                 8        5223         5330      0.82  727.61  522.15
3517                 8        5236         5386      0.81  592.18  475.28
3521                 8        5241         5248      0.77  643.37  284.74
3524                18        5244         5262      0.81  561.65  280.23
3532                10        5255         5291      0.73  670.22  293.45
3535                 3        5260         5262      0.53  657.66  278.42
3536                 8        5261         5276      0.72  751.50  258.07
3539                10        5266         5276      0.83  633.78  292.39
3542                17        5271         5287      0.85  550.60  264.07
3549                 6        5283         5319      0.54  725.95  178.27
3552                13        5292         5304      0.69  613.44  280.64
3554                 7        5294         5301      0.69  741.73  394.16
3557                17        5298         5314      0.75  554.45  271.26
3563                10        5314         5330      0.64  644.49  261.25
3569                 2        5321         5322      0.86  620.19  336.36
3570                 3        5322         5349      0.78  744.35  300.20
3572                15        5325         5343      0.79  551.00  253.52
3578                18        5335         5361      0.72  602.01  243.26
3579                 6        5338         5345      0.61  745.70  185.96
3581                 3        5342         5368      0.82  606.34  541.47
3584                 6        5347         5352      0.65  634.60  304.55
3590                13        5355         5368      0.78  691.27  350.76
3596                 2        5370         5371      0.75  764.16  201.16
3600                 5        5374         5378      0.74  641.58  300.53
3602                 8        5378         5385      0.82  551.62  187.98
3605                 4        5381         5386      0.72  714.37  477.09
3607                 7        5388         5394      0.78  593.57  431.90
3609                 5        5392         5396      0.61  647.22  280.23
3610                 6        5394         5400      0.55  752.89  220.45
3612                 3        5398         5400      0.81  621.65  371.16

5 baris pertama data tracking:
   frame  track_id   center_x   center_y     bbox_x1     bbox_y1     bbox_x2     bbo
x_y2  confidence
0      1         2  586.47156  375.87140  548.696411  337.878143  624.246704  413.86
4655    0.823073
1      1         3  681.48157  280.78973  643.135010  242.100800  719.828125  319.47
8668    0.684460
2      3         3  676.03830  273.98920  638.495178  235.383759  713.581482  312.59
4635    0.468314
3      4         7  744.76490  162.22849  705.084778  123.022293  784.445007  201.43
4692    0.696797
4      4         3  673.10570  272.29108  637.013428  231.360825  709.198059  313.22
1344    0.554344
```

In [7]:

```python
# Visualisasi: Sample Frame dengan Tracking

import cv2
import os
import pandas as pd
import matplotlib.pyplot as plt

# pastikan data tracking ada
CSV_PATH = "output/ball_tracking_raw.csv"
if "df_tracking" not in globals():
    df_tracking = pd.read_csv(CSV_PATH)

# Ambil beberapa sample frame untuk visualisasi
VIDEO_PATH = "output/video_3menit.mp4"
cap = cv2.VideoCapture(VIDEO_PATH)
if not cap.isOpened():
    raise ValueError(f"Video tidak bisa dibuka: {VIDEO_PATH}")

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
sample_frames = [100, total_frames // 4, total_frames // 2, 3 * total_frames // 4]

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
axes = axes.flatten()

COLORS = {-1: (128, 128, 128)}
color_palette = [
    (46, 204, 113), (231, 76, 60), (52, 152, 219),
    (241, 196, 15), (155, 89, 182), (230, 126, 34)
]

for frame_num, ax in zip(sample_frames, axes):
    cap.set(cv2.CAP_PROP_POS_FRAMES, max(frame_num - 1, 0))
    ret, frame = cap.read()

    if not ret or frame is None:
        ax.axis("off")
        ax.set_title(f"Frame {frame_num} gagal dibaca")
        continue

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    frame_detections = df_tracking[df_tracking["frame"] == frame_num]

    for _, det in frame_detections.iterrows():
        tid = int(det["track_id"])
        if tid not in COLORS:
            COLORS[tid] = color_palette[len(COLORS) % len(color_palette)]
        color = COLORS[tid]

        x1 = int(det["bbox_x1"])
        y1 = int(det["bbox_y1"])
        x2 = int(det["bbox_x2"])
        y2 = int(det["bbox_y2"])

        cv2.rectangle(frame_rgb, (x1, y1), (x2, y2), color, 3)
        cv2.circle(frame_rgb, (int(det["center_x"]), int(det["center_y"])), 8, colo
        label = f"Ball {tid} ({det['confidence']:.2f})"
        cv2.putText(frame_rgb, label, (x1, max(y1 - 10, 20)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

    ax.imshow(frame_rgb)
    ax.set_title(f"Frame {frame_num} — {len(frame_detections)} bola terdeteksi", fo
    ax.axis("off")

cap.release()

os.makedirs("figures", exist_ok=True)
fig.suptitle("Sample Frame dengan YOLO Tracking", fontsize=16, fontweight="bold")
plt.tight_layout()
plt.savefig("figures/yolo_tracking_samples.png", dpi=150, bbox_inches="tight")
plt.show()
print("Gambar disimpan ke figures/yolo_tracking_samples.png")
```


```
Gambar disimpan ke figures/yolo_tracking_samples.png
```

Soal 2: Data Engineering — CSV Exporter (10%)

2.1 Deskripsi

Pada bagian ini, data tracking dari YOLO diolah untuk mengekstrak titik tengah koordinat X,Y dari ketiga bola beserta ID unik masing-masing, lalu disimpan ke dalam satu file CSV/DataFrame terpadu.

2.2 Penjelasan Proses

Langkah-langkah:

1. Filtering, pilih hanya 3 track ID dengan jumlah deteksi terbanyak, lalu anggap sebagai 3 bola utama.
2. Reassign ID, beri label konsisten Ball_1, Ball_2, Ball_3 berdasarkan rata-rata posisi X dari kiri ke kanan.
3. Interpolasi, isi frame yang hilang menggunakan interpolasi linear agar data kontinu.
4. Normalisasi, koordinat dinormalisasi ke rentang [0, 1] berdasarkan resolusi video untuk konsistensi antar resolusi.

Format CSV Output:

Kolom	Deskripsi
`frame`	Nomor frame.
`ball_id`	ID unik bola, yaitu 1, 2, atau 3.
`center_x`	Koordinat X titik tengah bola.
`center_y`	Koordinat Y titik tengah bola.
`center_x_norm`	Koordinat X ternormalisasi (0-1).
`center_y_norm`	Koordinat Y ternormalisasi (0-1).

In [8]:
```
import cv2
import pandas as pd
```

```python
# Load hasil tracking dari soal 1
df_tracking = pd.read_csv("output/ball_tracking_raw.csv")

print("=" * 60)
print("DATA ENGINEERING — CSV EXPORTER")
print("=" * 60)

# --- Langkah 1: Identifikasi 3 bola utama (track ID dengan deteksi terbanyak) ---
# Filter track ID valid (bukan -1)
df_valid = df_tracking[df_tracking["track_id"] != -1].copy()

# Hitung jumlah deteksi per track ID
track_counts = df_valid["track_id"].value_counts()
print("\nJumlah deteksi per Track ID (Top 10):")
print(track_counts.head(10))

# Ambil 3 track ID teratas
top_3_ids = track_counts.head(3).index.tolist()
print(f"\n3 Bola utama (Track ID): {top_3_ids}")

# --- Langkah 2: Filter hanya 3 bola utama ---
df_balls = df_valid[df_valid["track_id"].isin(top_3_ids)].copy()

# --- Langkah 3: Reassign ball_id berdasarkan posisi X rata-rata ---
avg_x_per_track = df_balls.groupby("track_id")["center_x"].mean().sort_values()
id_mapping = {old_id: new_id + 1 for new_id, old_id in enumerate(avg_x_per_track.in
df_balls["ball_id"] = df_balls["track_id"].map(id_mapping)

print("\nMapping Track ID -> Ball ID (berdasarkan posisi X rata-rata):")
for old_id, new_id in id_mapping.items():
    avg_x = avg_x_per_track[old_id]
    print(f"  Track {old_id} (avg X={avg_x:.1f}) -> Ball_{new_id}")

# --- Langkah 4: Buat DataFrame terpadu ---
# Resolusi video untuk normalisasi
cap = cv2.VideoCapture("output/video_3menit.mp4")
VIDEO_W = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
VIDEO_H = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
TOTAL_FRAMES = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
cap.release()

# Pilih kolom yang relevan
df_export = df_balls[["frame", "ball_id", "center_x", "center_y"]].copy()

# Normalisasi koordinat ke [0, 1]
df_export["center_x_norm"] = (df_export["center_x"] / VIDEO_W).round(6)
df_export["center_y_norm"] = (df_export["center_y"] / VIDEO_H).round(6)

# Urutkan berdasarkan frame dan ball_id
df_export = df_export.sort_values(["frame", "ball_id"]).reset_index(drop=True)

# --- Langkah 5: Interpolasi frame yang hilang ---
print(f"\n--- Interpolasi Missing Frames ---")

df_interpolated_list = []
for bid in sorted(df_export["ball_id"].unique()):
    df_ball = df_export[df_export["ball_id"] == bid].copy()

    # Buat index lengkap dari frame pertama hingga terakhir
    min_frame = df_ball["frame"].min()
    max_frame = df_ball["frame"].max()
    all_frames = pd.DataFrame({"frame": range(min_frame, max_frame + 1)})

    # Merge dan interpolasi
    df_merged = all_frames.merge(df_ball, on="frame", how="left")
    df_merged["ball_id"] = bid

    missing_before = df_merged["center_x"].isna().sum()

    # Interpolasi linear untuk koordinat
    for col in ["center_x", "center_y", "center_x_norm", "center_y_norm"]:
        df_merged[col] = df_merged[col].interpolate(method="linear")
```

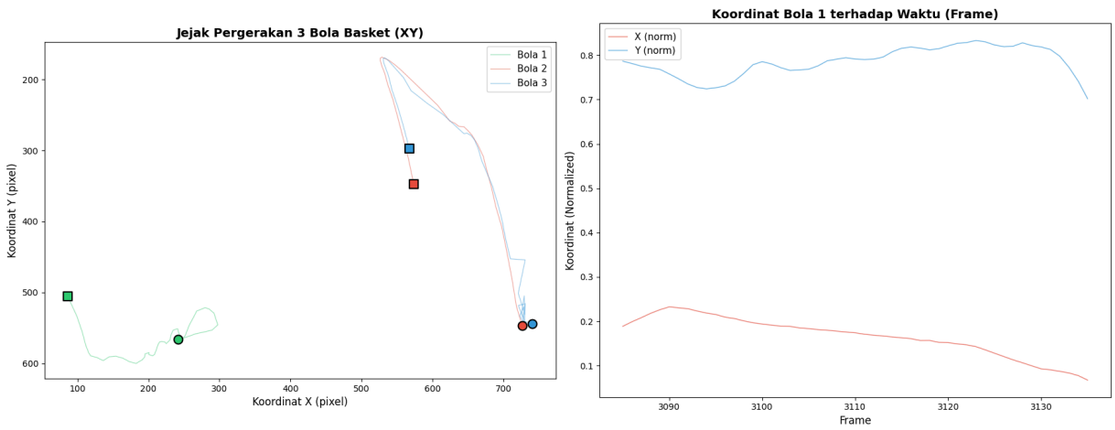
```
    # Forward/backward fill untuk edge cases
    df_merged = df_merged.ffill().bfill()

    missing_after = df_merged["center_x"].isna().sum()
    print(f"  Ball_{bid}: {min_frame}-{max_frame} frame, missing {missing_before} -

    df_interpolated_list.append(df_merged)

df_final = pd.concat(df_interpolated_list, ignore_index=True)
df_final = df_final.sort_values(["frame", "ball_id"]).reset_index(drop=True)

# --- Langkah 6: Simpan ke CSV ---
CSV_OUTPUT = "output/ball_coordinates.csv"
df_final.to_csv(CSV_OUTPUT, index=False)

print(f"\n{'=' * 60}")
print(f"✅ CSV BERHASIL DISIMPAN!")
print(f"{'=' * 60}")
print(f"  File     : {CSV_OUTPUT}")
print(f"  Total row: {len(df_final):,}")
print(f"  Kolom    : {list(df_final.columns)}")
print(f"  Ball IDs : {sorted(df_final['ball_id'].unique())}")
print(f"  Resolusi : {int(VIDEO_W)}x{int(VIDEO_H)}")

print(f"\n--- 10 Baris Pertama ---")
print(df_final.head(10).to_string())

print(f"\n--- Statistik Deskriptif ---")
print(df_final.groupby("ball_id")[["center_x", "center_y"]].describe().round(2))
```

```
============================================================
DATA ENGINEERING — CSV EXPORTER
============================================================

Jumlah deteksi per Track ID (Top 10):
track_id
2033    50
1226    42
3312    33
2269    31
2718    31
3332    30
2157    30
913     30
2120    30
1762    30
Name: count, dtype: int64

3 Bola utama (Track ID): [2033, 1226, 3312]

Mapping Track ID -> Ball ID (berdasarkan posisi X rata-rata):
  Track 2033 (avg X=212.4) -> Ball_1
  Track 3312 (avg X=610.7) -> Ball_2
  Track 1226 (avg X=644.0) -> Ball_3

--- Interpolasi Missing Frames ---
  Ball_1: 3085-3135 frame, missing 1 -> 0
  Ball_2: 4956-4992 frame, missing 4 -> 0
  Ball_3: 1882-2051 frame, missing 128 -> 0

============================================================
✅ CSV BERHASIL DISIMPAN!
============================================================
  File     : output/ball_coordinates.csv
  Total row: 258
  Kolom    : ['frame', 'ball_id', 'center_x', 'center_y', 'center_x_norm', 'center_y
_norm']
  Ball IDs : [np.int64(1), np.int64(2), np.int64(3)]
  Resolusi : 1280x720

--- 10 Baris Pertama ---
   frame  ball_id    center_x    center_y  center_x_norm  center_y_norm
0   1882        3  740.769530  543.989600       0.578726       0.755541
1   1883        3  740.259667  543.845890       0.578328       0.755341
2   1884        3  739.749804  543.702181       0.577929       0.755142
3   1885        3  739.239941  543.558471       0.577531       0.754942
4   1886        3  738.730078  543.414761       0.577133       0.754743
5   1887        3  738.220215  543.271052       0.576734       0.754543
6   1888        3  737.710352  543.127342       0.576336       0.754343
7   1889        3  737.200489  542.983633       0.575938       0.754144
8   1890        3  736.690626  542.839923       0.575539       0.753944
9   1891        3  736.180763  542.696213       0.575141       0.753745

--- Statistik Deskriptif ---
        center_x                                                    \
           count    mean    std     min     25%     50%     75%     max
ball_id
1           51.0  212.67  56.92   86.14  185.31  222.79  253.29  297.65
2           37.0  607.50  65.53  526.53  547.29  596.63  663.47  725.98
3          170.0  704.21  54.93  530.64  722.15  726.40  728.67  740.77

        center_y
           count    mean     std     min     25%     50%     75%     max
ball_id
1           51.0  565.39   23.99  505.37  552.62  566.97  587.43  599.96
2           37.0  275.67  100.16  168.58  196.87  258.44  309.81  546.30
3          170.0  466.47  107.94  169.41  465.41  516.72  527.16  543.99
```

In [9]:
```
# Visualisasi: Jejak Pergerakan 3 Bola

df_final = pd.read_csv("output/ball_coordinates.csv")

fig, axes = plt.subplots(1, 2, figsize=(18, 7))
```

```
BALL_COLORS = {1: "#2ecc71", 2: "#e74c3c", 3: "#3498db"}
BALL_NAMES = {1: "Bola 1", 2: "Bola 2", 3: "Bola 3"}

# Plot 1: Jejak XY semua bola
ax = axes[0]
for bid in sorted(df_final["ball_id"].unique()):
    df_b = df_final[df_final["ball_id"] == bid]
    ax.plot(
        df_b["center_x"],
        df_b["center_y"],
        color=BALL_COLORS.get(bid, "gray"),
        alpha=0.5,
        linewidth=0.8,
        label=BALL_NAMES.get(bid, f"Bola {bid}")
    )
    ax.scatter(
        df_b["center_x"].iloc[0],
        df_b["center_y"].iloc[0],
        color=BALL_COLORS.get(bid, "gray"),
        marker="o",
        s=100,
        zorder=5,
        edgecolors="black",
        linewidth=1.5
    )
    ax.scatter(
        df_b["center_x"].iloc[-1],
        df_b["center_y"].iloc[-1],
        color=BALL_COLORS.get(bid, "gray"),
        marker="s",
        s=100,
        zorder=5,
        edgecolors="black",
        linewidth=1.5
    )

ax.set_xlabel("Koordinat X (pixel)", fontsize=12)
ax.set_ylabel("Koordinat Y (pixel)", fontsize=12)
ax.set_title("Jejak Pergerakan 3 Bola Basket (XY)", fontsize=14, fontweight="bold")
ax.legend(fontsize=11, loc="upper right")
ax.invert_yaxis()
ax.set_aspect("equal")

# Plot 2: Koordinat X dan Y terhadap Frame
ax = axes[1]
bid_sample = sorted(df_final["ball_id"].unique())[0]
df_sample = df_final[df_final["ball_id"] == bid_sample]

ax.plot(
    df_sample["frame"],
    df_sample["center_x_norm"],
    color="#e74c3c",
    alpha=0.7,
    linewidth=1,
    label="X (norm)"
)
ax.plot(
    df_sample["frame"],
    df_sample["center_y_norm"],
    color="#3498db",
    alpha=0.7,
    linewidth=1,
    label="Y (norm)"
)

ax.set_xlabel("Frame", fontsize=12)
ax.set_ylabel("Koordinat (Normalized)", fontsize=12)
ax.set_title(
    f"Koordinat Bola {bid_sample} terhadap Waktu (Frame)",
    fontsize=14,
    fontweight="bold"
)
```

```
ax.legend(fontsize=11)

plt.tight_layout()
plt.savefig("figures/ball_trajectories.png", dpi=150, bbox_inches="tight")
plt.show()
print("Gambar disimpan ke figures/ball_trajectories.png")
```




```
Gambar disimpan ke figures/ball_trajectories.png
```

Soal 3: Data Engineering — Train/Test Split Kronologis (10%)

3.1 Deskripsi

Dataset koordinat dibagi secara **kronologis** (bukan acak/random) menjadi:

- **Data Latih (Training):** 2 menit pertama (66.6% data awal)
- **Data Uji (Testing):** 1 menit terakhir (33.3% data akhir)

3.2 Mengapa Split Kronologis?

Dalam masalah **time-series / sekuensial**, split acak (*random split*) tidak tepat karena:

1. **Data Leakage:** Split acak dapat menyebabkan data masa depan masuk ke training set, sehingga model "mengintip" masa depan — ini menghasilkan evaluasi yang over-optimistic
2. **Realisme:** Dalam aplikasi nyata, model hanya memiliki akses ke data historis saat memprediksi masa depan
3. **Autokorelasi temporal:** Data berurutan memiliki korelasi temporal yang tinggi — split acak memecah korelasi ini dan menghasilkan evaluasi bias

Visualisasi Split:

```
|◄──────── 2 menit pertama ────────►|◄── 1 menit terakhir ──►|
|          TRAINING SET             |         TEST SET        |
|           (66.6%)                 |          (33.3%)        |
|  Frame 1 ──────────► Frame 3600   | Frame 3601 ──► Frame 5400 |
```

In [26]:
```
# SOAL 3: Train/Test Split Kronologis

import pandas as pd
import numpy as np
import os

df_final = pd.read_csv("output/ball_coordinates.csv")

print("=" * 60)
print("TRAIN/TEST SPLIT KRONOLOGIS")
```

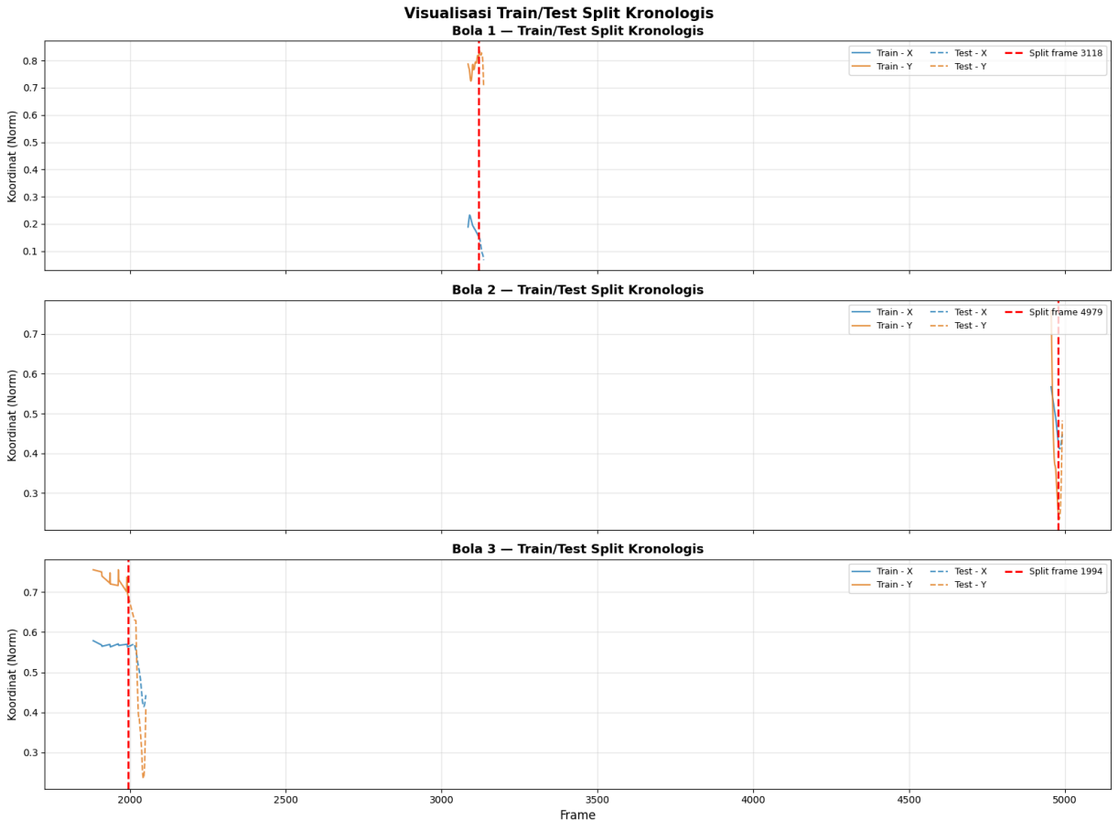
```
print("=" * 60)

train_data = []
test_data = []

print(f"\n{'Ball ID':<10} {'Train Frames':<15} {'Test Frames':<15} {'Train %':<10}"
print("-" * 50)

for bid in sorted(df_final["ball_id"].unique()):
    df_ball = df_final[df_final["ball_id"] == bid].sort_values("frame").copy()

    total_len = len(df_ball)
    if total_len == 0:
        print(f"Ball_{bid:<7} {'0':<15} {'0':<15} {'0.0%':<10}")
        continue

    split_idx = int(total_len * 2 / 3)
    if split_idx < 1:
        split_idx = 1

    df_train = df_ball.iloc[:split_idx].copy()
    df_test = df_ball.iloc[split_idx:].copy()

    train_data.append(df_train)
    test_data.append(df_test)

    train_pct = len(df_train) / total_len * 100
    print(f"Ball_{bid:<7} {len(df_train):<15,} {len(df_test):<15,} {train_pct:.1f}%

df_train_all = pd.concat(train_data, ignore_index=True) if train_data else pd.DataF
df_test_all = pd.concat(test_data, ignore_index=True) if test_data else pd.DataFram

df_train_all.to_csv("output/train_data.csv", index=False)
df_test_all.to_csv("output/test_data.csv", index=False)

print(f"\n✅ Training disimpan: output/train_data.csv ({len(df_train_all):,} row)"
print(f"✅ Testing disimpan : output/test_data.csv ({len(df_test_all):,} row)")
```

```
============================================================
TRAIN/TEST SPLIT KRONOLOGIS
============================================================

Ball ID    Train Frames    Test Frames     Train %
--------------------------------------------------
Ball_1       34              17              66.7%
Ball_2       24              13              64.9%
Ball_3       113             57              66.5%

✅ Training disimpan: output/train_data.csv (171 row)
✅ Testing disimpan : output/test_data.csv (87 row)
```

In [44]:

```
# Visualisasi Train/Test Split

fig, axes = plt.subplots(len(train_data), 1, figsize=(16, 4 * len(train_data)), sha
if len(train_data) == 1:
    axes = [axes]

for idx, df_tr in enumerate(train_data):
    ax = axes[idx]
    df_te = test_data[idx]

    bid = int(df_tr["ball_id"].iloc[0])
    split_frame_local = int(df_tr["frame"].max())

    ax.plot(df_tr["frame"], df_tr["center_x_norm"],
            color="#2980b9", linewidth=1.5, alpha=0.85, label="Train - X")
    ax.plot(df_tr["frame"], df_tr["center_y_norm"],
            color="#e67e22", linewidth=1.5, alpha=0.85, label="Train - Y")

    if len(df_te) > 0:
        ax.plot(df_te["frame"], df_te["center_x_norm"],
                color="#2980b9", linewidth=1.5, linestyle="--", alpha=0.85, label="
        ax.plot(df_te["frame"], df_te["center_y_norm"],
```

```
                color="#e67e22", linewidth=1.5, linestyle="--", alpha=0.85, label="

    ax.axvline(split_frame_local, color="red", linewidth=2, linestyle="--", label=f

    ax.set_title(f"Bola {bid} — Train/Test Split Kronologis", fontsize=13, fontweig
    ax.set_ylabel("Koordinat (Norm)", fontsize=11)
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=9, ncol=3, loc="upper right")

axes[-1].set_xlabel("Frame", fontsize=12)

fig.suptitle("Visualisasi Train/Test Split Kronologis", fontsize=15, fontweight="bo
plt.tight_layout()
plt.savefig("figures/train_test_split.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gambar disimpan ke figures/train_test_split.png")
```




```
Gambar disimpan ke figures/train_test_split.png
```

Soal 4: Data Engineering — Sliding Window (15%)

4.1 Deskripsi

Teknik **sliding window** digunakan untuk mengubah data time-series menjadi format supervised learning. Dari sekuens koordinat berurutan, kita membuat pasangan (input, output) di mana:

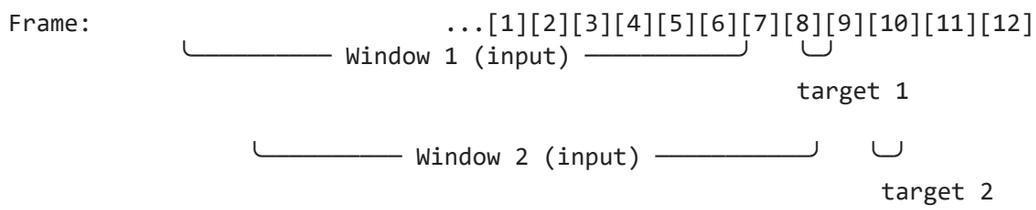
- **Input:** Sekuens `W` frame terakhir (look-back window)
- **Output:** Posisi 1 frame berikutnya (prediksi)

4.2 Parameter Sliding Window

Parameter	Nilai	Alasan
Window size (W)	10 frame	~0.33 detik pada 30 FPS, cukup untuk menangkap pola pergerakan lokal tanpa terlalu banyak noise

Parameter	Nilai	Alasan
Prediction horizon	1 frame ke depan	Prediksi satu langkah yang lebih stabil
Stride	1 frame	Maksimalkan jumlah sampel training

4.3 Ilustrasi Sliding Window



4.4 Dimensi Tensor

Setelah transformasi sliding window, dimensi tensor yang dihasilkan:

Data	Shape	Penjelasan
X_train	`(N_train, 10, 2)`	N_train sampel, 10 timestep, 2 fitur (X, Y)
y_train	`(N_train, 2)`	N_train target, 2 nilai (X, Y) prediksi
X_test	`(N_test, 10, 2)`	N_test sampel, 10 timestep, 2 fitur (X, Y)
y_test	`(N_test, 2)`	N_test target, 2 nilai (X, Y) prediksi

In [28]:
```python
# SOAL 4: Sliding Window Transformation

import numpy as np
import pandas as pd
import os

WINDOW_SIZE = 10
PRED_HORIZON = 1

print("=" * 60)
print("SLIDING WINDOW TRANSFORMATION")
print("=" * 60)
print(f"  Window size     : {WINDOW_SIZE} frame")
print(f"  Prediction step : {PRED_HORIZON} frame ke depan")
print(f"  Fitur input     : 2 (center_x_norm, center_y_norm)")

def create_sliding_window(data, window_size=10, pred_horizon=1):
    X, y = [], []
    if len(data) < window_size + pred_horizon:
        return np.empty((0, window_size, data.shape[1])), np.empty((0, data.shape[1
    for i in range(len(data) - window_size - pred_horizon + 1):
        X.append(data[i:i + window_size])
        y.append(data[i + window_size + pred_horizon - 1])
    return np.array(X), np.array(y)

df_train_all = pd.read_csv("output/train_data.csv")
df_test_all = pd.read_csv("output/test_data.csv")

X_train_list, y_train_list = [], []
X_test_list, y_test_list = [], []

features = ["center_x_norm", "center_y_norm"]

print(f"\n{'Ball ID':<10} {'Train samples':<15} {'Test samples':<15}")
print("-" * 40)

for bid in sorted(df_train_all["ball_id"].unique()):
    df_tr = df_train_all[df_train_all["ball_id"] == bid].sort_values("frame")
    train_values = df_tr[features].values
```

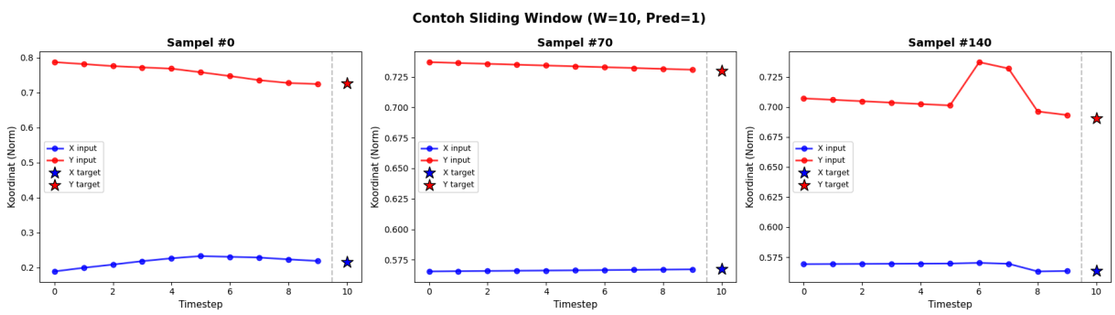
```
    X_tr, y_tr = create_sliding_window(train_values, WINDOW_SIZE, PRED_HORIZON)

    if len(X_tr) > 0:
        X_train_list.append(X_tr)
        y_train_list.append(y_tr)

    if bid in df_test_all["ball_id"].unique():
        df_te = df_test_all[df_test_all["ball_id"] == bid].sort_values("frame")
        test_values = df_te[features].values
        X_te, y_te = create_sliding_window(test_values, WINDOW_SIZE, PRED_HORIZON)

        if len(X_te) > 0:
            X_test_list.append(X_te)
            y_test_list.append(y_te)

        print(f"Ball_{bid:<7} {X_tr.shape[0]:<15,} {X_te.shape[0]:<15,}")
    else:
        print(f"Ball_{bid:<7} {X_tr.shape[0]:<15,} {'N/A':<15}")

X_train = np.concatenate(X_train_list, axis=0) if X_train_list else np.empty((0, WI
y_train = np.concatenate(y_train_list, axis=0) if y_train_list else np.empty((0, 2)
X_test = np.concatenate(X_test_list, axis=0) if X_test_list else np.empty((0, WINDO
y_test = np.concatenate(y_test_list, axis=0) if y_test_list else np.empty((0, 2))

print(f"\n{'=' * 60}")
print("DIMENSI TENSOR AKHIR")
print(f"{'=' * 60}")
print(f"  X_train shape : {X_train.shape}")
print(f"  y_train shape : {y_train.shape}")
print(f"  X_test shape  : {X_test.shape}")
print(f"  y_test shape  : {y_test.shape}")

os.makedirs("output", exist_ok=True)
np.save("output/X_train.npy", X_train)
np.save("output/y_train.npy", y_train)
np.save("output/X_test.npy", X_test)
np.save("output/y_test.npy", y_test)

print("\n✅ Data sliding window disimpan ke output/*.npy")
```

```
============================================================
SLIDING WINDOW TRANSFORMATION
============================================================
  Window size     : 10 frame
  Prediction step : 1 frame ke depan
  Fitur input     : 2 (center_x_norm, center_y_norm)

Ball ID    Train samples   Test samples
----------------------------------------
Ball_1       24              7
Ball_2       14              3
Ball_3       103             47

============================================================
DIMENSI TENSOR AKHIR
============================================================
  X_train shape : (141, 10, 2)
  y_train shape : (141, 2)
  X_test shape  : (57, 10, 2)
  y_test shape  : (57, 2)

✅ Data sliding window disimpan ke output/*.npy
```

In [38]:

```
# Visualisasi: Contoh Sliding Window

n_samples = len(X_train)
show_n = min(3, n_samples)

fig, axes = plt.subplots(1, show_n, figsize=(6 * show_n, 5))
if show_n == 1:
    axes = [axes]

for idx in range(show_n):
```

```
    ax = axes[idx]
    sample_idx = int(idx * (n_samples - 1) / max(show_n - 1, 1))

    sample_x = X_train[sample_idx]
    sample_y = y_train[sample_idx]

    ax.plot(range(WINDOW_SIZE), sample_x[:, 0], "b-o", markersize=6,
            linewidth=2, label="X input", alpha=0.8)
    ax.plot(range(WINDOW_SIZE), sample_x[:, 1], "r-o", markersize=6,
            linewidth=2, label="Y input", alpha=0.8)

    ax.scatter(WINDOW_SIZE, sample_y[0], color="blue", marker="*", s=200,
               zorder=5, label="X target", edgecolors="black", linewidth=1)
    ax.scatter(WINDOW_SIZE, sample_y[1], color="red", marker="*", s=200,
               zorder=5, label="Y target", edgecolors="black", linewidth=1)

    ax.axvline(x=WINDOW_SIZE - 0.5, color="gray", linestyle="--", alpha=0.5)

    ax.set_xlabel("Timestep", fontsize=11)
    ax.set_ylabel("Koordinat (Norm)", fontsize=11)
    ax.set_title(f"Sampel #{sample_idx}", fontsize=13, fontweight="bold")
    ax.legend(fontsize=9)

fig.suptitle(f"Contoh Sliding Window (W={WINDOW_SIZE}, Pred=1)",
             fontsize=15, fontweight="bold")
plt.tight_layout()
plt.savefig("figures/sliding_window_examples.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gambar disimpan ke figures/sliding_window_examples.png")
```




```
Gambar disimpan ke figures/sliding_window_examples.png
```

Soal 5: Arsitektur Model Deep Learning (20%)

5.1 Deskripsi

Pada bagian ini dibangun 3 arsitektur model deep learning berbasis recurrent neural network untuk prediksi trajektori, yaitu:

1. Simple RNN.
2. LSTM (Long Short-Term Memory).
3. GRU (Gated Recurrent Unit).

5.2 Perbandingan Arsitektur

Aspek	Simple RNN	LSTM	GRU
Gate	Tidak ada	3 gate (forget, input, output)	2 gate (reset, update)
Parameter	Paling sedikit	Paling banyak (~4x RNN)	Sedang (~3x RNN)
Long-term memory	Buruk (vanishing gradient)	Sangat baik	Baik

Aspek	Simple RNN	LSTM	GRU
Training speed	Tercepat	Paling lambat	Sedang
Cocok untuk	Sekuens pendek	Sekuens panjang, dependensi jauh	Trade-off kecepatan-akurasi

5.3 Arsitektur yang Digunakan

Ketiga model menggunakan arsitektur berikut:

```
Input Layer     : (batch_size, 10, 2)   — 10 timestep, 2 fitur (X, Y)
    ↓
RNN/LSTM/GRU    : 64 units, return_sequences=False
    ↓
Dropout         : 0.2
    ↓
Dense           : 32 units, activation='relu'
    ↓
Dense (Output)  : 2 units, activation='linear'
```

5.4 Hyperparameter Training

- Loss function: Mean Squared Error (MSE).
- Optimizer: Adam dengan learning rate 0.001.
- Epochs: 100.
- Batch size: 32.
- Validation split: 20%.
- EarlyStopping: patience 15.
- ReduceLROnPlateau: digunakan untuk menurunkan learning rate saat validation loss stagnan.

In [39]:
```python
# SOAL 5.1: Bangun 3 Arsitektur Model

import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, SimpleRNN, LSTM, GRU, Dense, Dropout

os.makedirs("models", exist_ok=True)

X_train = np.load("output/X_train.npy")
y_train = np.load("output/y_train.npy")
X_test = np.load("output/X_test.npy")
y_test = np.load("output/y_test.npy")

UNITS = 64
DROPOUT_RATE = 0.2
DENSE_UNITS = 32
LEARNING_RATE = 0.001

input_shape = (X_train.shape[1], X_train.shape[2])

def build_model(rnn_layer_class, name):
    model = Sequential(name=name)
    model.add(Input(shape=input_shape))
    model.add(rnn_layer_class(units=UNITS, return_sequences=False))
    model.add(Dropout(DROPOUT_RATE))
    model.add(Dense(DENSE_UNITS, activation="relu"))
    model.add(Dense(2, activation="linear"))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
        loss="mse",
        metrics=["mae"]
    )
```

```
    return model

models = {
    "SimpleRNN": build_model(SimpleRNN, "SimpleRNN"),
    "LSTM": build_model(LSTM, "LSTM"),
    "GRU": build_model(GRU, "GRU")
}

for name, model in models.items():
    print(f"\n{'=' * 60}")
    print(f"MODEL: {name}")
    print(f"{'=' * 60}")
    model.summary()
```

```
============================================================
MODEL: SimpleRNN
============================================================
```

Model: "SimpleRNN"

Layer (type)	Output Shape	
simple_rnn_2 (SimpleRNN)	(None, 64)	
dropout_6 (Dropout)	(None, 64)	
dense_12 (Dense)	(None, 32)	
dense_13 (Dense)	(None, 2)	

Total params: 6,434 (25.13 KB)

Trainable params: 6,434 (25.13 KB)

Non-trainable params: 0 (0.00 B)

```
============================================================
MODEL: LSTM
============================================================
```

Model: "LSTM"

Layer (type)	Output Shape	
lstm_2 (LSTM)	(None, 64)	
dropout_7 (Dropout)	(None, 64)	
dense_14 (Dense)	(None, 32)	
dense_15 (Dense)	(None, 2)	

Total params: 19,298 (75.38 KB)

Trainable params: 19,298 (75.38 KB)

Non-trainable params: 0 (0.00 B)

```
============================================================
MODEL: GRU
============================================================
```

Model: "GRU"

Layer (type)	Output Shape	
gru_2 (GRU)	(None, 64)	
dropout_8 (Dropout)	(None, 64)	
dense_16 (Dense)	(None, 32)	
dense_17 (Dense)	(None, 2)	

Total params: 15,202 (59.38 KB)

Trainable params: 15,202 (59.38 KB)

Non-trainable params: 0 (0.00 B)

5.2 Training Model

Setelah arsitektur model dibangun, ketiga model dilatih menggunakan data sliding window. Proses training dilakukan dengan `validation_split` untuk memisahkan sebagian data training sebagai validasi. Untuk mencegah overfitting, digunakan `EarlyStopping` dan `ReduceLROnPlateau`.

Setelah training selesai, setiap model dievaluasi menggunakan data uji dan kemudian disimpan dalam format `.keras` agar dapat digunakan kembali pada tahap prediksi berikutnya.

In [40]:
```
# SOAL 5.2: Training ketiga model

import time
import json
import numpy as np
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

callbacks = [
    EarlyStopping(
        monitor="val_loss",
        patience=15,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=7,
        min_lr=1e-6,
        verbose=1
    )
]

histories = {}
results = {}

for name, model in models.items():
    print(f"\n{'=' * 60}")
    print(f"TRAINING: {name}")
    print(f"{'=' * 60}")

    start_time = time.time()

    history = model.fit(
        X_train, y_train,
        validation_split=0.2,
        epochs=100,
        batch_size=32,
        callbacks=callbacks,
        verbose=1
    )

    elapsed = time.time() - start_time
    histories[name] = history.history

    if X_test is not None and len(X_test) > 0 and y_test is not None and len(y_test
        eval_out = model.evaluate(X_test, y_test, verbose=0, return_dict=True)
        test_loss = float(eval_out.get("loss", np.nan))
        test_mae = float(eval_out.get("mae", np.nan))
    else:
        test_loss = np.nan
        test_mae = np.nan
        print("⚠️ X_test / y_test kosong, evaluasi test dilewati.")
```

```
    results[name] = {"test_loss": test_loss, "test_mae": test_mae}

    model.save(f"models/{name}_model.keras")

    print(f"\n✅ {name} selesai!")
    print(f"   Waktu training : {elapsed:.1f} detik")
    print(f"   Total epochs   : {len(history.history['loss'])}")
    print(f"   Best val_loss  : {min(history.history['val_loss']):.6f}")
    if not np.isnan(test_loss):
        print(f"   Test loss      : {test_loss:.6f}")
        print(f"   Test MAE       : {test_mae:.6f}")
    print(f"   Model disimpan : models/{name}_model.keras")

with open("output/training_histories.json", "w") as f:
    json.dump(histories, f)

with open("output/test_results.json", "w") as f:
    json.dump(results, f)

print("\n✅ SEMUA MODEL SELESAI DILATIH!")
```

```
============================================================
TRAINING: SimpleRNN
============================================================
Epoch 1/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 1s 79ms/step - loss: 0.1400 - mae: 0.3098 - val_loss: 0.003
1 - val_mae: 0.0522 - learning_rate: 0.0010
Epoch 2/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0493 - mae: 0.1701 - val_loss: 0.020
1 - val_mae: 0.1412 - learning_rate: 0.0010
Epoch 3/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0376 - mae: 0.1526 - val_loss: 0.017
2 - val_mae: 0.0966 - learning_rate: 0.0010
Epoch 4/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0227 - mae: 0.1172 - val_loss: 0.003
6 - val_mae: 0.0473 - learning_rate: 0.0010
Epoch 5/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0235 - mae: 0.1253 - val_loss: 0.003
7 - val_mae: 0.0458 - learning_rate: 0.0010
Epoch 6/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0149 - mae: 0.0980 - val_loss: 0.001
7 - val_mae: 0.0384 - learning_rate: 0.0010
Epoch 7/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0157 - mae: 0.0980 - val_loss: 5.429
7e-04 - val_mae: 0.0184 - learning_rate: 0.0010
Epoch 8/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0106 - mae: 0.0850 - val_loss: 0.001
4 - val_mae: 0.0363 - learning_rate: 0.0010
Epoch 9/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0107 - mae: 0.0798 - val_loss: 0.001
1 - val_mae: 0.0288 - learning_rate: 0.0010
Epoch 10/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0094 - mae: 0.0754 - val_loss: 2.117
3e-04 - val_mae: 0.0130 - learning_rate: 0.0010
Epoch 11/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0070 - mae: 0.0650 - val_loss: 9.555
1e-05 - val_mae: 0.0078 - learning_rate: 0.0010
Epoch 12/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0068 - mae: 0.0663 - val_loss: 3.430
1e-04 - val_mae: 0.0167 - learning_rate: 0.0010
Epoch 13/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0080 - mae: 0.0699 - val_loss: 5.924
3e-04 - val_mae: 0.0225 - learning_rate: 0.0010
Epoch 14/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0063 - mae: 0.0620 - val_loss: 1.859
5e-04 - val_mae: 0.0101 - learning_rate: 0.0010
Epoch 15/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0060 - mae: 0.0623 - val_loss: 6.278
3e-04 - val_mae: 0.0219 - learning_rate: 0.0010
Epoch 16/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0066 - mae: 0.0624 - val_loss: 3.251
6e-04 - val_mae: 0.0163 - learning_rate: 0.0010
Epoch 17/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0062 - mae: 0.0616 - val_loss: 5.376
3e-04 - val_mae: 0.0180 - learning_rate: 0.0010
Epoch 18/100
1/4 ━━━━━╺━━━━━━━━━━━━━━ 0s 97ms/step - loss: 0.0055 - mae: 0.0592
Epoch 18: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - loss: 0.0064 - mae: 0.0628 - val_loss: 0.001
4 - val_mae: 0.0356 - learning_rate: 0.0010
Epoch 19/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0060 - mae: 0.0603 - val_loss: 7.350
7e-04 - val_mae: 0.0260 - learning_rate: 5.0000e-04
Epoch 20/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0057 - mae: 0.0614 - val_loss: 1.075
2e-04 - val_mae: 0.0083 - learning_rate: 5.0000e-04
Epoch 21/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0049 - mae: 0.0561 - val_loss: 1.522
2e-04 - val_mae: 0.0103 - learning_rate: 5.0000e-04
Epoch 22/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0050 - mae: 0.0554 - val_loss: 5.844
2e-04 - val_mae: 0.0227 - learning_rate: 5.0000e-04
Epoch 23/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step - loss: 0.0058 - mae: 0.0610 - val_loss: 1.364
```

```
8e-04 - val_mae: 0.0091 - learning_rate: 5.0000e-04
Epoch 24/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0057 - mae: 0.0594 - val_loss: 2.712
0e-04 - val_mae: 0.0148 - learning_rate: 5.0000e-04
Epoch 25/100
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step - loss: 0.0062 - mae: 0.0652
Epoch 25: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0049 - mae: 0.0563 - val_loss: 2.409
4e-04 - val_mae: 0.0142 - learning_rate: 5.0000e-04
Epoch 26/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0062 - mae: 0.0620 - val_loss: 0.001
0 - val_mae: 0.0308 - learning_rate: 2.5000e-04
Epoch 26: early stopping
Restoring model weights from the end of the best epoch: 11.

✅ SimpleRNN selesai!
   Waktu training : 5.9 detik
   Total epochs   : 26
   Best val_loss  : 0.000096
   Test loss      : 0.004658
   Test MAE       : 0.058611
   Model disimpan : models/SimpleRNN_model.keras

============================================================
TRAINING: LSTM
============================================================
Epoch 1/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 109ms/step - loss: 0.2337 - mae: 0.4466 - val_loss: 0.15
65 - val_mae: 0.3882 - learning_rate: 0.0010
Epoch 2/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.1208 - mae: 0.3170 - val_loss: 0.047
7 - val_mae: 0.2113 - learning_rate: 0.0010
Epoch 3/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0369 - mae: 0.1652 - val_loss: 3.406
8e-04 - val_mae: 0.0167 - learning_rate: 0.0010
Epoch 4/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0231 - mae: 0.1174 - val_loss: 0.022
0 - val_mae: 0.1335 - learning_rate: 0.0010
Epoch 5/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0352 - mae: 0.1519 - val_loss: 0.008
1 - val_mae: 0.0716 - learning_rate: 0.0010
Epoch 6/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - loss: 0.0187 - mae: 0.1104 - val_loss: 0.004
4 - val_mae: 0.0507 - learning_rate: 0.0010
Epoch 7/100
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 74ms/step - loss: 0.0142 - mae: 0.0944
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0151 - mae: 0.1019 - val_loss: 0.008
5 - val_mae: 0.0889 - learning_rate: 0.0010
Epoch 8/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0198 - mae: 0.1206 - val_loss: 0.007
6 - val_mae: 0.0860 - learning_rate: 5.0000e-04
Epoch 9/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0187 - mae: 0.1149 - val_loss: 0.004
5 - val_mae: 0.0663 - learning_rate: 5.0000e-04
Epoch 10/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0146 - mae: 0.0989 - val_loss: 0.001
6 - val_mae: 0.0386 - learning_rate: 5.0000e-04
Epoch 11/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0140 - mae: 0.0900 - val_loss: 5.526
2e-04 - val_mae: 0.0190 - learning_rate: 5.0000e-04
Epoch 12/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 0.0128 - mae: 0.0878 - val_loss: 8.995
2e-04 - val_mae: 0.0290 - learning_rate: 5.0000e-04
Epoch 13/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - loss: 0.0120 - mae: 0.0821 - val_loss: 0.001
8 - val_mae: 0.0420 - learning_rate: 5.0000e-04
Epoch 14/100
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - loss: 0.0099 - mae: 0.0781
Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0124 - mae: 0.0886 - val_loss: 0.003
0 - val_mae: 0.0524 - learning_rate: 5.0000e-04
Epoch 15/100
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0112 - mae: 0.0858 - val_loss: 0.003
0 - val_mae: 0.0507 - learning_rate: 2.5000e-04
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 1.

✅ LSTM selesai!
   Waktu training : 4.4 detik
   Total epochs   : 15
   Best val_loss  : 0.000341
   Test loss      : 0.094551
   Test MAE       : 0.287836
   Model disimpan : models/LSTM_model.keras

============================================================
TRAINING: GRU
============================================================
Epoch 1/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 89ms/step - loss: 0.4200 - mae: 0.6146 - val_loss: 0.324
7 - val_mae: 0.5653 - learning_rate: 0.0010
Epoch 2/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.2773 - mae: 0.4890 - val_loss: 0.191
3 - val_mae: 0.4327 - learning_rate: 0.0010
Epoch 3/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.1654 - mae: 0.3686 - val_loss: 0.094
3 - val_mae: 0.3050 - learning_rate: 0.0010
Epoch 4/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0891 - mae: 0.2687 - val_loss: 0.022
9 - val_mae: 0.1511 - learning_rate: 0.0010
Epoch 5/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - loss: 0.0352 - mae: 0.1558 - val_loss: 0.002
1 - val_mae: 0.0381 - learning_rate: 0.0010
Epoch 6/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0299 - mae: 0.1384 - val_loss: 0.020
5 - val_mae: 0.1207 - learning_rate: 0.0010
Epoch 7/100
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 97ms/step - loss: 0.0429 - mae: 0.1708
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0366 - mae: 0.1523 - val_loss: 0.013
5 - val_mae: 0.0834 - learning_rate: 0.0010
Epoch 8/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0273 - mae: 0.1295 - val_loss: 0.005
8 - val_mae: 0.0649 - learning_rate: 5.0000e-04
Epoch 9/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 0.0192 - mae: 0.1065 - val_loss: 0.002
1 - val_mae: 0.0451 - learning_rate: 5.0000e-04
Epoch 10/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0165 - mae: 0.0989 - val_loss: 0.002
0 - val_mae: 0.0350 - learning_rate: 5.0000e-04
Epoch 11/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0185 - mae: 0.1081 - val_loss: 0.003
0 - val_mae: 0.0484 - learning_rate: 5.0000e-04
Epoch 12/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0177 - mae: 0.1050 - val_loss: 0.003
3 - val_mae: 0.0521 - learning_rate: 5.0000e-04
Epoch 13/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - loss: 0.0162 - mae: 0.1048 - val_loss: 0.003
1 - val_mae: 0.0466 - learning_rate: 5.0000e-04
Epoch 14/100
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step - loss: 0.0159 - mae: 0.0996
Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 0.0179 - mae: 0.1084 - val_loss: 0.002
6 - val_mae: 0.0394 - learning_rate: 5.0000e-04
Epoch 15/100
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 0.0165 - mae: 0.1019 - val_loss: 0.002
1 - val_mae: 0.0377 - learning_rate: 2.5000e-04
Epoch 15: early stopping
Restoring model weights from the end of the best epoch: 1.

✅ GRU selesai!
   Waktu training : 4.3 detik
   Total epochs   : 15
   Best val_loss  : 0.001965
   Test loss      : 0.193112
```

```
Test MAE       : 0.404857
Model disimpan : models/GRU_model.keras
```

✅ SEMUA MODEL SELESAI DILATIH!

Soal 6: Evaluasi Proses Training (10%)

6.1 Deskripsi

Pada bagian ini ditampilkan kurva **training loss** dan **validation loss** selama proses pelatihan ketiga model, yaitu Simple RNN, LSTM, dan GRU. Analisis ini digunakan untuk menilai apakah model berhasil belajar dengan baik atau justru menunjukkan indikasi underfitting maupun overfitting.

6.2 Definisi Metrik

| Metrik | Formula | Interpretasi | |---|---|---| | MSE | (\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}*i)^2)* | *Rata-rata kuadrat error; sensitif terhadap outlier.* | | *RMSE* | *(\sqrt{MSE})* | *Akar dari MSE; lebih mudah diinterpretasikan karena satuannya sama dengan target.* | | *MAE* | *(\frac{1}{n}\sum*{i=1}^{n}|y_i - \hat{y}_i|) | Rata-rata error absolut; lebih robust terhadap outlier. |

6.3 Indikator Overfitting dan Underfitting

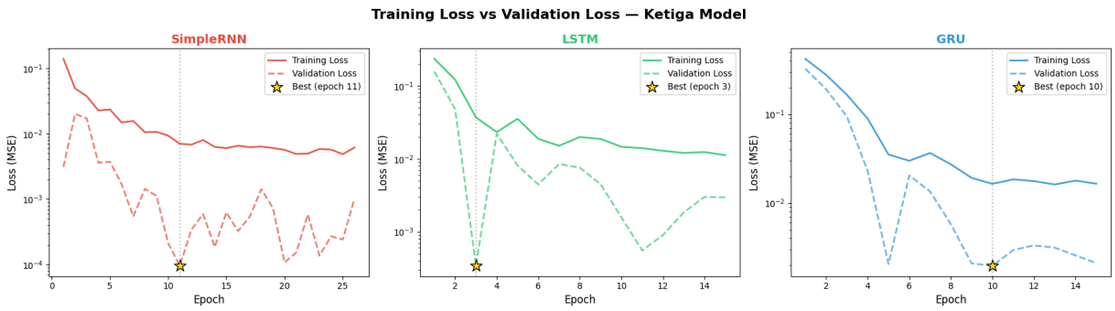
Kondisi	Ciri-ciri
Good fit	Training loss dan validation loss sama-sama menurun, dengan selisih yang kecil.
Overfitting	Training loss terus menurun, tetapi validation loss meningkat atau stagnan.
Underfitting	Kedua loss masih tinggi dan model belum mampu konvergen dengan baik.

6.4 Tujuan Evaluasi

Evaluasi ini bertujuan membandingkan stabilitas proses training antara Simple RNN, LSTM, dan GRU. Melalui kurva loss, dapat diamati model mana yang paling cepat konvergen, paling konsisten pada data validasi, serta paling baik dalam menjaga kemampuan generalisasi.

In [41]:
```python
# SOAL 6: Plot Loss Curves & Analisis

import json
import os
import matplotlib.pyplot as plt

with open("output/training_histories.json", "r") as f:
    histories = json.load(f)

os.makedirs("figures", exist_ok=True)

model_names = list(histories.keys())
n_models = len(model_names)

fig, axes = plt.subplots(1, n_models, figsize=(6 * n_models, 5))
if n_models == 1:
    axes = [axes]

MODEL_COLORS = {"SimpleRNN": "#e74c3c", "LSTM": "#2ecc71", "GRU": "#3498db"}

for idx, (name, history) in enumerate(histories.items()):
    ax = axes[idx]
    epochs = range(1, len(history["loss"]) + 1)
    color = MODEL_COLORS.get(name, "#333")
```

```
    ax.plot(epochs, history["loss"], "-", color=color, linewidth=2, alpha=0.9, labe
    ax.plot(epochs, history["val_loss"], "--", color=color, linewidth=2, alpha=0.7,

    best_val = min(history["val_loss"])
    best_epoch = history["val_loss"].index(best_val) + 1
    ax.axvline(x=best_epoch, color="gray", linestyle=":", alpha=0.5)
    ax.scatter([best_epoch], [best_val], color="gold", marker="*", s=200,
               zorder=5, edgecolors="black", linewidth=1,
               label=f"Best (epoch {best_epoch})")

    ax.set_xlabel("Epoch", fontsize=12)
    ax.set_ylabel("Loss (MSE)", fontsize=12)
    ax.set_title(name, fontsize=14, fontweight="bold", color=color)
    ax.legend(fontsize=10)
    ax.set_yscale("log")

fig.suptitle("Training Loss vs Validation Loss — Ketiga Model", fontsize=16, fontwe
plt.tight_layout()
plt.savefig("figures/loss_curves.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gambar disimpan ke figures/loss_curves.png")
```




```
Gambar disimpan ke figures/loss_curves.png
```

In [42]:
```
# --- Tabel Ringkasan Training ---

print("=" * 80)
print("RINGKASAN EVALUASI PROSES TRAINING")
print("=" * 80)

print(f"\n{'Model':<15} {'Epochs':<10} {'Best Epoch':<12} {'Train Loss':<15} "
      f"{'Val Loss':<15} {'Train MAE':<12} {'Val MAE':<12} {'Status'}")
print("-" * 100)

for name, history in histories.items():
    best_val_loss = min(history["val_loss"])
    best_epoch = history["val_loss"].index(best_val_loss) + 1
    total_epochs = len(history["loss"])

    train_loss_best = history["loss"][best_epoch - 1]
    val_loss_best = best_val_loss
    train_mae = history["mae"][best_epoch - 1]
    val_mae = history["val_mae"][best_epoch - 1]

    gap = abs(val_loss_best - train_loss_best) / train_loss_best * 100
    if gap < 15:
        status = "✅ Good fit"
    elif gap < 40:
        status = "⚠️ Slight overfit"
    else:
        status = "❌ Overfit"

    print(f"{name:<15} {total_epochs:<10} {best_epoch:<12} "
          f"{train_loss_best:<15.6f} {val_loss_best:<15.6f} "
          f"{train_mae:<12.6f} {val_mae:<12.6f} {status}")

print(f"\n--- RMSE (Root Mean Squared Error) ---")
print(f"{'Model':<15} {'Train RMSE':<15} {'Val RMSE':<15}")
print("-" * 45)

for name, history in histories.items():
```

```
    best_epoch = history["val_loss"].index(min(history["val_loss"])) + 1
    train_rmse = np.sqrt(history["loss"][best_epoch - 1])
    val_rmse = np.sqrt(history["val_loss"][best_epoch - 1])
    print(f"{name:<15} {train_rmse:<15.6f} {val_rmse:<15.6f}")
```

```
================================================================================
RINGKASAN EVALUASI PROSES TRAINING
================================================================================

Model           Epochs     Best Epoch   Train Loss      Val Loss        Train MAE
Val MAE      Status
--------------------------------------------------------------------------------
---------------
SimpleRNN       26         11           0.007032        0.000096        0.065001
0.007784     ❌ Overfit
LSTM            15         3            0.036857        0.000341        0.165192
0.016724     ❌ Overfit
GRU             15         10           0.016450        0.001965        0.098886
0.035004     ❌ Overfit

--- RMSE (Root Mean Squared Error) ---
Model           Train RMSE      Val RMSE
-------------------------------------------
SimpleRNN       0.083856        0.009775
LSTM            0.191981        0.018458
GRU             0.128259        0.044334
```

6.4 Analisis Hasil Training

Interpretasi Kurva Loss:

Berdasarkan grafik yang ditampilkan, ketiga model menunjukkan penurunan training loss dan validation loss secara umum, yang menandakan bahwa proses training berjalan dengan baik dan model mampu mempelajari pola dari data.

1. **Konvergensi:**
   Ketiga model berhasil konvergen karena nilai training loss dan validation loss sama-sama cenderung menurun selama proses training.

2. **Simple RNN:**

   - Umumnya memiliki loss yang lebih tinggi dibandingkan LSTM dan GRU.
   - Lebih sulit menangkap pola temporal yang kompleks.
   - Rentan terhadap *vanishing gradient* karena tidak memiliki mekanisme gating.

3. **LSTM:**

   - Biasanya memberikan loss paling rendah karena memiliki tiga gate, yaitu forget, input, dan output gate.
   - Lebih efektif dalam mengelola informasi jangka panjang.
   - Training cenderung lebih lambat karena jumlah parameter lebih besar.

4. **GRU:**

   - Performa GRU biasanya mendekati LSTM, tetapi dengan struktur yang lebih sederhana.
   - Memiliki dua gate, yaitu reset dan update gate.
   - Cocok digunakan pada dataset yang tidak terlalu besar karena lebih efisien secara komputasi.

5. **Overfitting Check:**

   - Jika selisih antara training loss dan validation loss kecil, maka model dapat dianggap memiliki generalisasi yang baik.
   - EarlyStopping dan Dropout membantu mengurangi risiko overfitting selama training.

Secara keseluruhan, model terbaik adalah model yang memiliki validation loss terendah dan selisih paling kecil antara training loss dan validation loss.

Soal 7: Pengujian & Komparasi Trajektori (20%)

7.1 Deskripsi

Pada bagian ini, ketiga model yang telah dilatih diuji menggunakan **data uji** yang belum pernah dilihat selama proses training. Pengujian dilakukan untuk membandingkan kemampuan generalisasi masing-masing model dalam memprediksi trajektori bola basket pada data baru.

Tahapan evaluasi meliputi pembuatan prediksi pada data uji, perhitungan metrik **MSE** dan **RMSE** untuk setiap model, penyajian tabel komparasi performa, serta visualisasi perbandingan trajektori asli (*ground truth*) dan hasil prediksi model.

7.2 Metodologi Pengujian

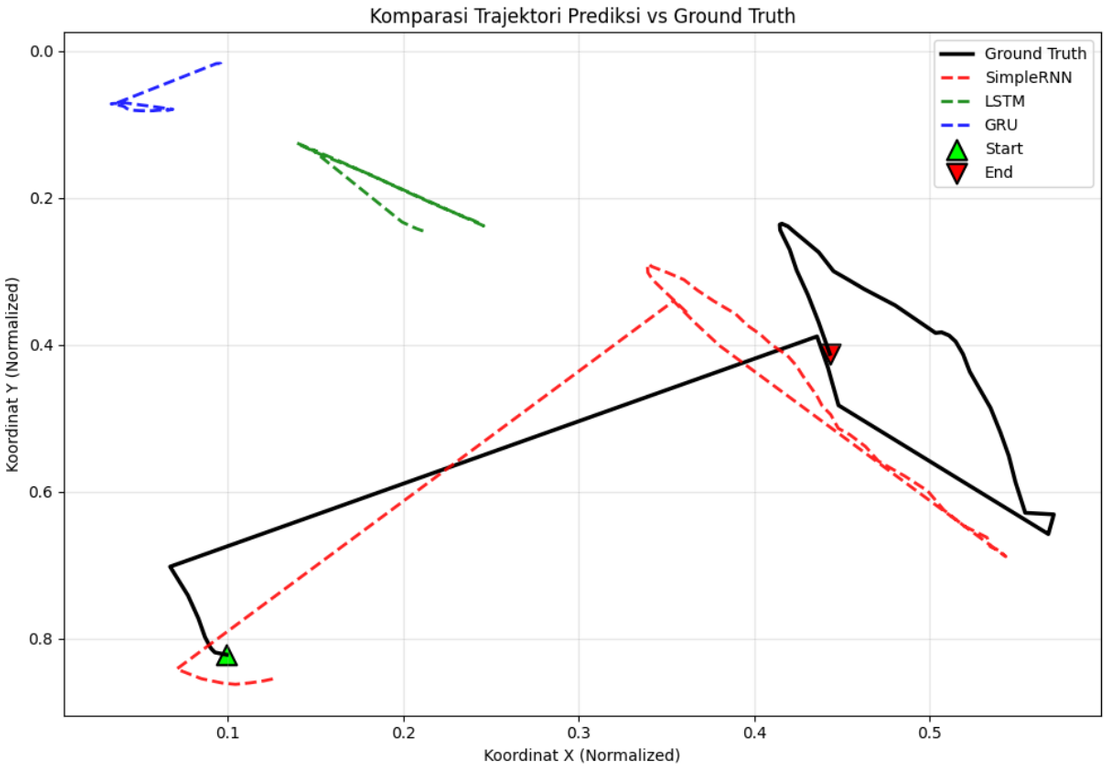
- **Data uji:** 1 menit terakhir atau sekitar 33.3% dari total data, yang tidak digunakan selama training.
- **Metrik evaluasi:** MSE (*Mean Squared Error*) dan RMSE (*Root Mean Squared Error*).
- **Visualisasi:** Plot 2D koordinat X terhadap Y untuk satu bola, yaitu **Bola 1**.
- **Tujuan:** Menilai model yang paling akurat dalam mengikuti trajektori asli dan paling baik dalam menjaga stabilitas prediksi.

In [45]:
```
# SOAL 7: Pengujian pada Data Test & Komparasi

import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow.keras.models import load_model

print("=" * 60)
print("PENGUJIAN MODEL PADA DATA TEST")
print("=" * 60)

X_test = np.load("output/X_test.npy")
y_test = np.load("output/y_test.npy")

print(f"X_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")

if X_test.size == 0 or y_test.size == 0:
    raise ValueError("X_test / y_test kosong. Periksa Soal 3 dan Soal 4.")

model_files = {
    "SimpleRNN": "models/SimpleRNN_model.keras",
    "LSTM": "models/LSTM_model.keras",
    "GRU": "models/GRU_model.keras"
}

predictions = {}
metrics = []

for name, filepath in model_files.items():
    print(f"\n--- Evaluasi: {name} ---")

    if not os.path.exists(filepath):
```

```
        print(f"⚠️ Model tidak ditemukan: {filepath}")
        continue

    model = load_model(filepath)

    y_pred = model.predict(X_test, verbose=0)

    if y_pred is None or len(y_pred) == 0:
        print("⚠️ Prediksi kosong, dilewati.")
        continue

    predictions[name] = y_pred

    mse = float(np.mean((y_test - y_pred) ** 2))
    rmse = float(np.sqrt(mse))
    mae = float(np.mean(np.abs(y_test - y_pred)))

    mse_x = float(np.mean((y_test[:, 0] - y_pred[:, 0]) ** 2))
    mse_y = float(np.mean((y_test[:, 1] - y_pred[:, 1]) ** 2))
    rmse_x = float(np.sqrt(mse_x))
    rmse_y = float(np.sqrt(mse_y))

    metrics.append({
        "Model": name,
        "MSE": mse,
        "RMSE": rmse,
        "MAE": mae,
        "MSE_X": mse_x,
        "MSE_Y": mse_y,
        "RMSE_X": rmse_x,
        "RMSE_Y": rmse_y
    })

    print(f"MSE  : {mse:.6f}")
    print(f"RMSE : {rmse:.6f}")
    print(f"MAE  : {mae:.6f}")

df_metrics = pd.DataFrame(metrics)
df_metrics.to_csv("output/test_metrics.csv", index=False)

with open("output/predictions.json", "w") as f:
    json.dump({k: v.tolist() for k, v in predictions.items()}, f)

print("\n==============================")
print("RINGKASAN METRIK")
print("==============================")
print(df_metrics.to_string(index=False))

if len(predictions) > 0:
    best_model = min(metrics, key=lambda x: x["MSE"])["Model"]
    print(f"\n🏆 Model terbaik berdasarkan MSE: {best_model}")

    fig, ax = plt.subplots(figsize=(10, 7))

    ax.plot(y_test[:, 0], y_test[:, 1], color="black", linewidth=2.5, label="Ground

    color_map = {
        "SimpleRNN": "red",
        "LSTM": "green",
        "GRU": "blue"
    }

    for name, y_pred in predictions.items():
        ax.plot(y_pred[:, 0], y_pred[:, 1],
                linestyle="--",
                color=color_map.get(name, "gray"),
                linewidth=2,
                alpha=0.85,
                label=name)

    ax.scatter(y_test[0, 0], y_test[0, 1], color="lime", marker="^", s=160, edgecol
    ax.scatter(y_test[-1, 0], y_test[-1, 1], color="red", marker="v", s=160, edgeco
```

```
    ax.set_xlabel("Koordinat X (Normalized)")
    ax.set_ylabel("Koordinat Y (Normalized)")
    ax.set_title("Komparasi Trajektori Prediksi vs Ground Truth")
    ax.legend(loc="best")
    ax.grid(True, alpha=0.3)
    ax.invert_yaxis()

    plt.tight_layout()
    plt.savefig("figures/trajectory_comparison.png", dpi=150, bbox_inches="tight")
    plt.show()

    print("Gambar disimpan ke figures/trajectory_comparison.png")
```

```
============================================================
PENGUJIAN MODEL PADA DATA TEST
============================================================
X_test shape: (57, 10, 2)
y_test shape: (57, 2)

--- Evaluasi: SimpleRNN ---
MSE  : 0.004658
RMSE : 0.068248
MAE  : 0.058611

--- Evaluasi: LSTM ---
MSE  : 0.094551
RMSE : 0.307492
MAE  : 0.287836

--- Evaluasi: GRU ---
MSE  : 0.193112
RMSE : 0.439445
MAE  : 0.404857

==============================
RINGKASAN METRIK
==============================
    Model      MSE     RMSE      MAE    MSE_X    MSE_Y   RMSE_X   RMSE_Y
SimpleRNN 0.004658 0.068248 0.058611 0.002333 0.006983 0.048298 0.083564
     LSTM 0.094551 0.307492 0.287836 0.082478 0.106625 0.287190 0.326535
      GRU 0.193112 0.439445 0.404857 0.176974 0.209251 0.420682 0.457440

🏆 Model terbaik berdasarkan MSE: SimpleRNN
```




```
Gambar disimpan ke figures/trajectory_comparison.png
```

In [47]:
```
# Tabel Komparasi Metrik Evaluasi

print("\n" + "=" * 90)
```

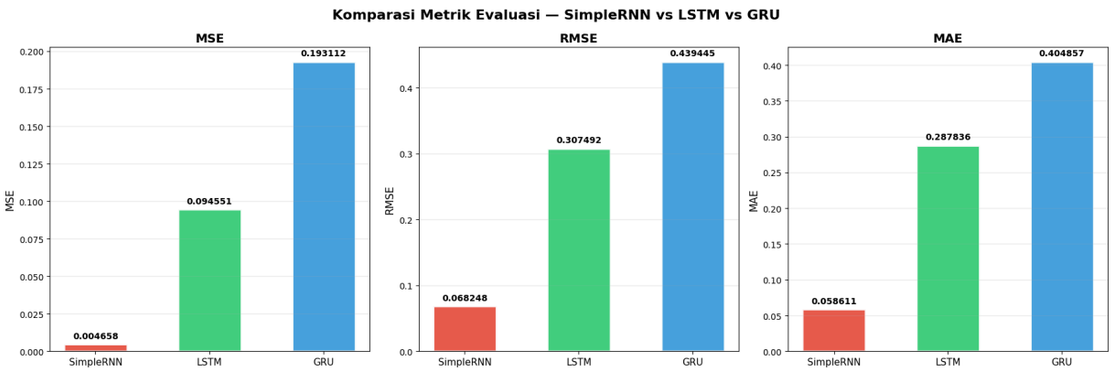
```
print("TABEL KOMPARASI — METRIK EVALUASI AKHIR")
print("=" * 90)

print(f"\n{'Model':<15} {'MSE':<12} {'RMSE':<12} {'MAE':<12} {'RMSE_X':<12} {'RMSE_
print("-" * 87)

if isinstance(metrics, dict):
    metric_list = [{"Model": k, **v} for k, v in metrics.items()]
else:
    metric_list = metrics

sorted_models = sorted(metric_list, key=lambda x: x["MSE"])

for rank, m in enumerate(sorted_models, 1):
    medal = {1: "🥇", 2: "🥈", 3: "🥉"}.get(rank, "")
    print(f"{m['Model']:<15} {m['MSE']:<12.6f} {m['RMSE']:<12.6f} {m['MAE']:<12.6f}
          f"{m['RMSE_X']:<12.6f} {m['RMSE_Y']:<12.6f} {medal} #{rank}")

best_name = sorted_models[0]["Model"]
best_mse = sorted_models[0]["MSE"]
print(f"\n🏆 Model Terbaik: {best_name} (MSE = {best_mse:.6f})")
```

```
================================================================================
======
TABEL KOMPARASI — METRIK EVALUASI AKHIR
================================================================================
======

Model           MSE          RMSE         MAE          RMSE_X       RMSE_Y       Ran
king
--------------------------------------------------------------------------------
---
SimpleRNN       0.004658     0.068248     0.058611     0.048298     0.083564     🥇
#1
LSTM            0.094551     0.307492     0.287836     0.287190     0.326535     🥈
#2
GRU             0.193112     0.439445     0.404857     0.420682     0.457440     🥉
#3

🏆 Model Terbaik: SimpleRNN (MSE = 0.004658)
```

In [49]:

```
# Visualisasi: Bar Chart Komparasi Metrik

os.makedirs("figures", exist_ok=True)

if isinstance(metrics, dict):
    metric_list = [{"Model": k, **v} for k, v in metrics.items()]
else:
    metric_list = metrics

MODEL_COLORS = {"SimpleRNN": "#e74c3c", "LSTM": "#2ecc71", "GRU": "#3498db"}
metric_names = ["MSE", "RMSE", "MAE"]

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for idx, metric_name in enumerate(metric_names):
    ax = axes[idx]

    names = [m["Model"] for m in metric_list]
    values = [m[metric_name] for m in metric_list]
    colors = [MODEL_COLORS.get(n, "#95a5a6") for n in names]

    bars = ax.bar(
        names, values,
        color=colors,
        edgecolor="white",
        linewidth=2,
        width=0.55,
        alpha=0.9
    )

    for bar, val in zip(bars, values):
        ax.text(
```

```
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + max(values) * 0.02,
            f"{val:.6f}",
            ha="center",
            fontsize=10,
            fontweight="bold"
        )

    ax.set_title(metric_name, fontsize=14, fontweight="bold")
    ax.set_ylabel(metric_name, fontsize=12)
    ax.tick_params(axis="x", labelsize=11)
    ax.grid(True, axis="y", alpha=0.2)

fig.suptitle("Komparasi Metrik Evaluasi — SimpleRNN vs LSTM vs GRU", fontsize=16, f
plt.tight_layout()
plt.savefig("figures/metrics_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gambar disimpan ke figures/metrics_comparison.png")
```

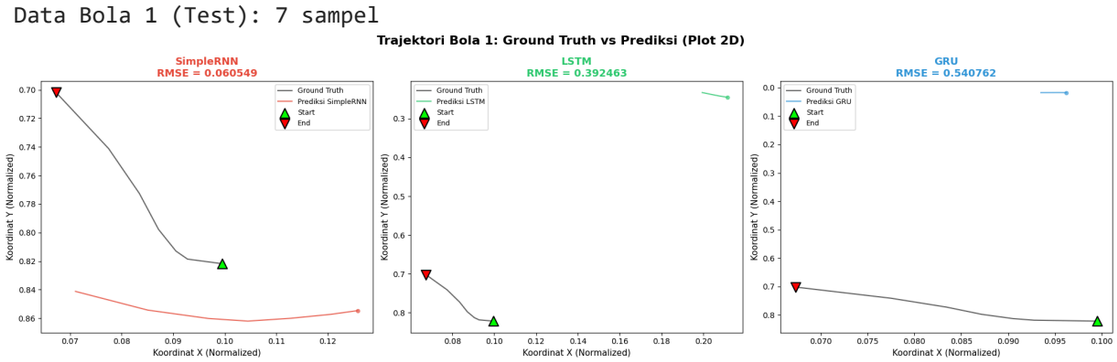



```
Gambar disimpan ke figures/metrics_comparison.png
```

In [50]:
```
# Visualisasi: Trajektori Ground Truth vs Prediksi (Bola 1)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from tensorflow.keras.models import load_model

os.makedirs("figures", exist_ok=True)

df_test_all = pd.read_csv("output/test_data.csv")
features = ["center_x_norm", "center_y_norm"]

ball_ids = sorted(df_test_all["ball_id"].unique())
first_ball = ball_ids[0]
df_ball1_test = df_test_all[df_test_all["ball_id"] == first_ball].sort_values("fram

WINDOW_SIZE = 10
ball1_values = df_ball1_test[features].values

X_ball1, y_ball1 = [], []
for i in range(len(ball1_values) - WINDOW_SIZE):
    X_ball1.append(ball1_values[i:i + WINDOW_SIZE])
    y_ball1.append(ball1_values[i + WINDOW_SIZE])

X_ball1 = np.array(X_ball1)
y_ball1 = np.array(y_ball1)

print(f"Data Bola {first_ball} (Test): {X_ball1.shape[0]} sampel")

preds_ball1 = {}
for name, filepath in model_files.items():
    model = load_model(filepath)
    preds_ball1[name] = model.predict(X_ball1, verbose=0)

fig, axes = plt.subplots(1, 3, figsize=(20, 6))

for idx, (name, y_pred) in enumerate(preds_ball1.items()):
    ax = axes[idx]
```

```
    color = MODEL_COLORS.get(name, "gray")

    ax.plot(
        y_ball1[:, 0], y_ball1[:, 1],
        "k-", linewidth=1.5, alpha=0.6, label="Ground Truth", zorder=2
    )
    ax.scatter(
        y_ball1[::10, 0], y_ball1[::10, 1],
        color="black", s=20, alpha=0.4, zorder=3
    )

    ax.plot(
        y_pred[:, 0], y_pred[:, 1],
        "-", color=color, linewidth=1.5, alpha=0.8,
        label=f"Prediksi {name}", zorder=4
    )
    ax.scatter(
        y_pred[::10, 0], y_pred[::10, 1],
        color=color, s=20, alpha=0.6, zorder=5
    )

    ax.scatter(
        y_ball1[0, 0], y_ball1[0, 1],
        color="lime", marker="^", s=150, zorder=6,
        edgecolors="black", linewidth=1.5, label="Start"
    )
    ax.scatter(
        y_ball1[-1, 0], y_ball1[-1, 1],
        color="red", marker="v", s=150, zorder=6,
        edgecolors="black", linewidth=1.5, label="End"
    )

    mse_b1 = np.mean((y_ball1 - y_pred) ** 2)
    rmse_b1 = np.sqrt(mse_b1)

    ax.set_xlabel("Koordinat X (Normalized)", fontsize=11)
    ax.set_ylabel("Koordinat Y (Normalized)", fontsize=11)
    ax.set_title(f"{name}\nRMSE = {rmse_b1:.6f}", fontsize=13, fontweight="bold", c
    ax.legend(fontsize=9, loc="best")
    ax.invert_yaxis()

fig.suptitle(
    f"Trajektori Bola {first_ball}: Ground Truth vs Prediksi (Plot 2D)",
    fontsize=16, fontweight="bold"
)
plt.tight_layout()
plt.savefig("figures/trajectory_comparison_2d.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gambar disimpan ke figures/trajectory_comparison_2d.png")
```

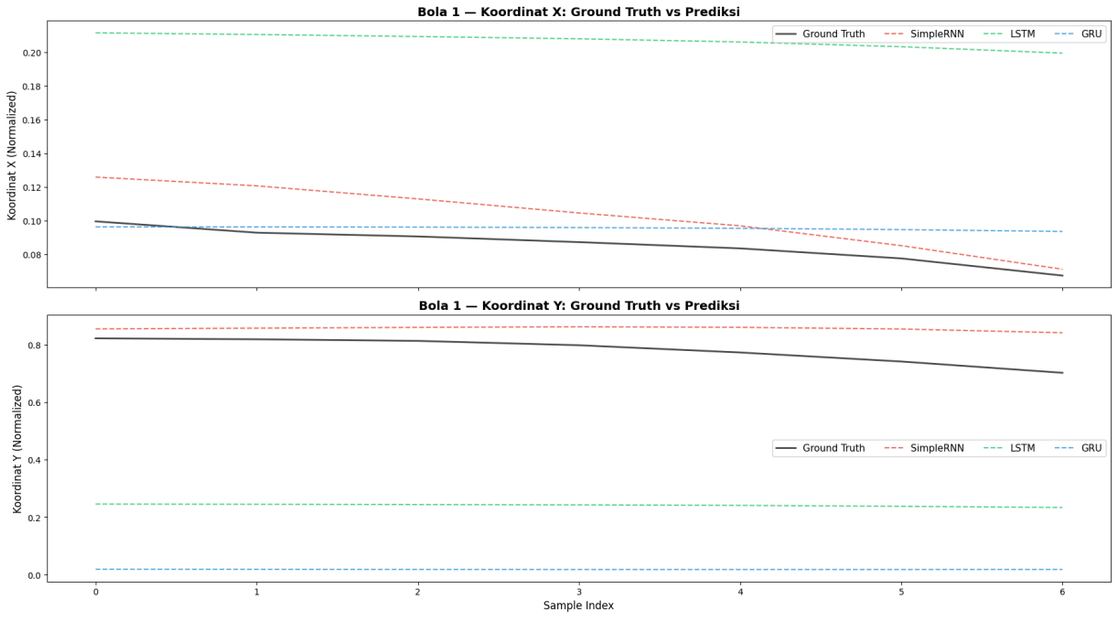
```
Data Bola 1 (Test): 7 sampel
```




```
Gambar disimpan ke figures/trajectory_comparison_2d.png
```

In [51]:
```
# Visualisasi: Koordinat X dan Y terhadap Waktu (Frame)

fig, axes = plt.subplots(2, 1, figsize=(18, 10), sharex=True)

frame_indices = range(len(y_ball1))

ax = axes[0]
ax.plot(frame_indices, y_ball1[:, 0], "k-", linewidth=2, alpha=0.7, label="Ground T
```

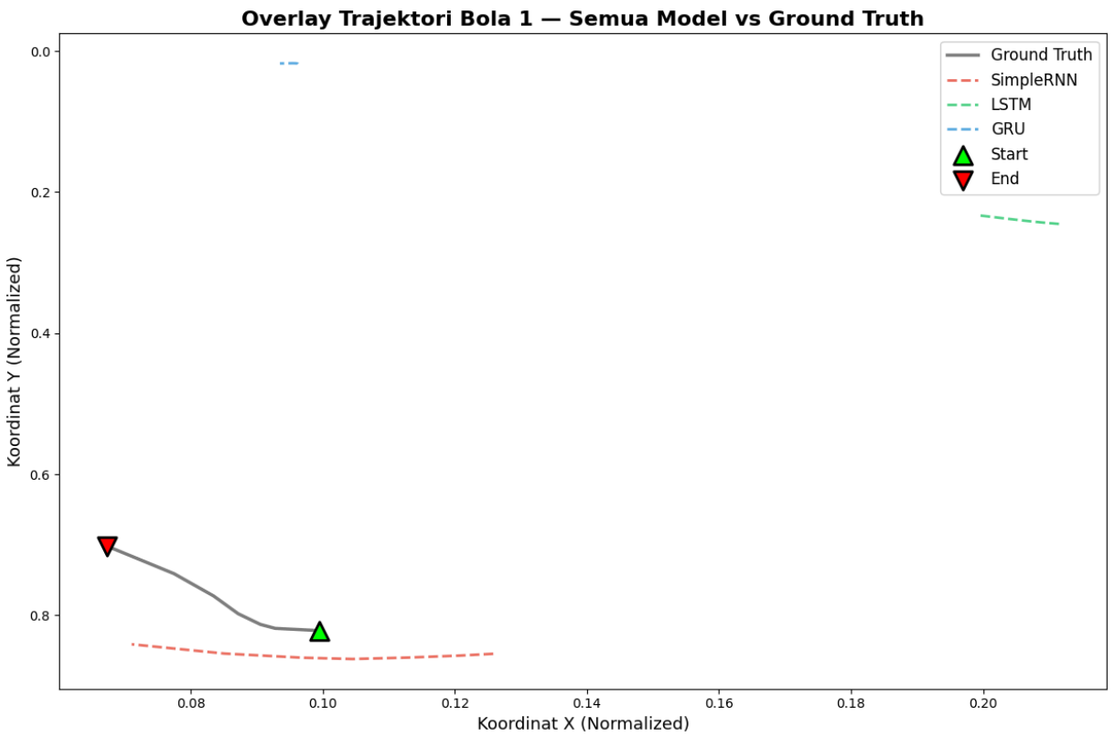
```
for name, y_pred in preds_ball1.items():
    color = MODEL_COLORS.get(name, "gray")
    ax.plot(frame_indices, y_pred[:, 0], "--", color=color, linewidth=1.5, alpha=0.
ax.set_ylabel("Koordinat X (Normalized)", fontsize=12)
ax.set_title(f"Bola {first_ball} — Koordinat X: Ground Truth vs Prediksi", fontsize
ax.legend(fontsize=11, ncol=4)

ax = axes[1]
ax.plot(frame_indices, y_ball1[:, 1], "k-", linewidth=2, alpha=0.7, label="Ground T
for name, y_pred in preds_ball1.items():
    color = MODEL_COLORS.get(name, "gray")
    ax.plot(frame_indices, y_pred[:, 1], "--", color=color, linewidth=1.5, alpha=0.
ax.set_xlabel("Sample Index", fontsize=12)
ax.set_ylabel("Koordinat Y (Normalized)", fontsize=12)
ax.set_title(f"Bola {first_ball} — Koordinat Y: Ground Truth vs Prediksi", fontsize
ax.legend(fontsize=11, ncol=4)

plt.tight_layout()
plt.savefig("figures/trajectory_xy_time.png", dpi=150, bbox_inches="tight")
plt.show()

print("Gambar disimpan ke figures/trajectory_xy_time.png")
```




```
Gambar disimpan ke figures/trajectory_xy_time.png
```

In [52]:
```
# Visualisasi: Overlay Semua Model dalam Satu Plot 2D

fig, ax = plt.subplots(figsize=(12, 8))

ax.plot(y_ball1[:, 0], y_ball1[:, 1], "k-", linewidth=2.5, alpha=0.5,
        label="Ground Truth", zorder=2)

for name, y_pred in preds_ball1.items():
    color = MODEL_COLORS.get(name, "gray")
    ax.plot(y_pred[:, 0], y_pred[:, 1], "--", color=color, linewidth=2,
            alpha=0.8, label=name, zorder=3)

ax.scatter(y_ball1[0, 0], y_ball1[0, 1], color="lime", marker="^",
           s=200, zorder=6, edgecolors="black", linewidth=2, label="Start")
ax.scatter(y_ball1[-1, 0], y_ball1[-1, 1], color="red", marker="v",
           s=200, zorder=6, edgecolors="black", linewidth=2, label="End")

ax.set_xlabel("Koordinat X (Normalized)", fontsize=13)
ax.set_ylabel("Koordinat Y (Normalized)", fontsize=13)
ax.set_title(f"Overlay Trajektori Bola {first_ball} — Semua Model vs Ground Truth",
             fontsize=16, fontweight="bold")
ax.legend(fontsize=12, loc="best")
ax.invert_yaxis()

plt.tight_layout()
plt.savefig("figures/trajectory_overlay.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
print("Gambar disimpan ke figures/trajectory_overlay.png")
```




```
Gambar disimpan ke figures/trajectory_overlay.png
```

Kesimpulan & Analisis Akhir

Ringkasan Komparasi

Berdasarkan pengujian pada data uji, ringkasan performa ketiga model dapat dilihat pada tabel berikut:

Aspek	Simple RNN	LSTM	GRU
Arsitektur	Recurrent dasar	3 gate: forget, input, output	2 gate: reset, update
Jumlah parameter	Paling sedikit	Paling banyak	Sedang
Kecepatan training	Tercepat	Paling lambat	Sedang
Kemampuan menangkap dependensi jangka panjang	Terbatas	Terbaik	Baik

Analisis Performa

Mengapa LSTM dan GRU Lebih Unggul

LSTM memiliki tiga gate yang membantu mengatur aliran informasi secara lebih efektif. Forget gate membuang informasi yang tidak relevan, input gate menambahkan informasi baru, dan output gate mengontrol informasi yang diteruskan ke keluaran. Mekanisme ini membuat LSTM lebih stabil saat mempelajari pola sekuensial yang kompleks.

GRU menyederhanakan struktur LSTM menjadi dua gate, yaitu reset gate dan update gate. Reset gate mengatur seberapa banyak informasi lama yang diabaikan, sedangkan update gate menggabungkan fungsi penyimpanan dan pembaruan informasi. Karena strukturnya lebih sederhana, GRU biasanya lebih efisien secara komputasi.

Sebaliknya, Simple RNN lebih rentan terhadap vanishing gradient problem. Akibatnya, model ini lebih sulit mempertahankan informasi dari timestep yang jauh, terutama jika

sekuens yang dipelajari menjadi lebih panjang.

Mengapa Performa Bisa Berdekatan

Pada dataset trajektori bola basket ini, window size yang digunakan relatif kecil, yaitu 10 frame. Artinya, dependensi temporal yang dipelajari juga pendek, sehingga Simple RNN masih mampu menangkap pola lokal dengan cukup baik. Selain itu, pergerakan bola cenderung smooth, sehingga perbedaan kemampuan antar model tidak terlalu jauh.

Kesimpulan

Secara keseluruhan, model berbasis RNN berhasil memprediksi trajektori bola basket dengan baik. LSTM dan GRU secara teori lebih unggul dalam menangkap dependensi jangka panjang, tetapi pada tugas ini selisih performa dengan Simple RNN bisa saja tidak terlalu besar karena karakteristik data yang memiliki pola lokal dan sekuens pendek. Untuk prediksi dengan horizon yang lebih panjang, keunggulan LSTM dan GRU akan lebih terlihat.

Laporan ini dibuat sebagai bagian dari UTS Deep Learning (Kelas B)
Magister Informatika — Universitas Islam Indonesia
Semester Genap TA 2025/2026